

コンピュータ アーキテクチャ I 第6回

塩谷 亮太 (shioya@ci.i.u-tokyo.ac.jp)

東京大学大学院情報理工学系研究科 創造情報学専攻

課題の解説

課題 5

- 第4回の講義資料を参考に、P型/N型リレーを使って以下を構成せよ

- 2入力 OR ゲート
- 3入力 NOR ゲート

2入力 OR の真理値表

<i>a</i>	<i>b</i>	<i>o</i>
0	0	0
0	1	1
1	0	1
1	1	1

3入力 NOR の真理値表

<i>a</i>	<i>b</i>	<i>c</i>	<i>o</i>
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

方針

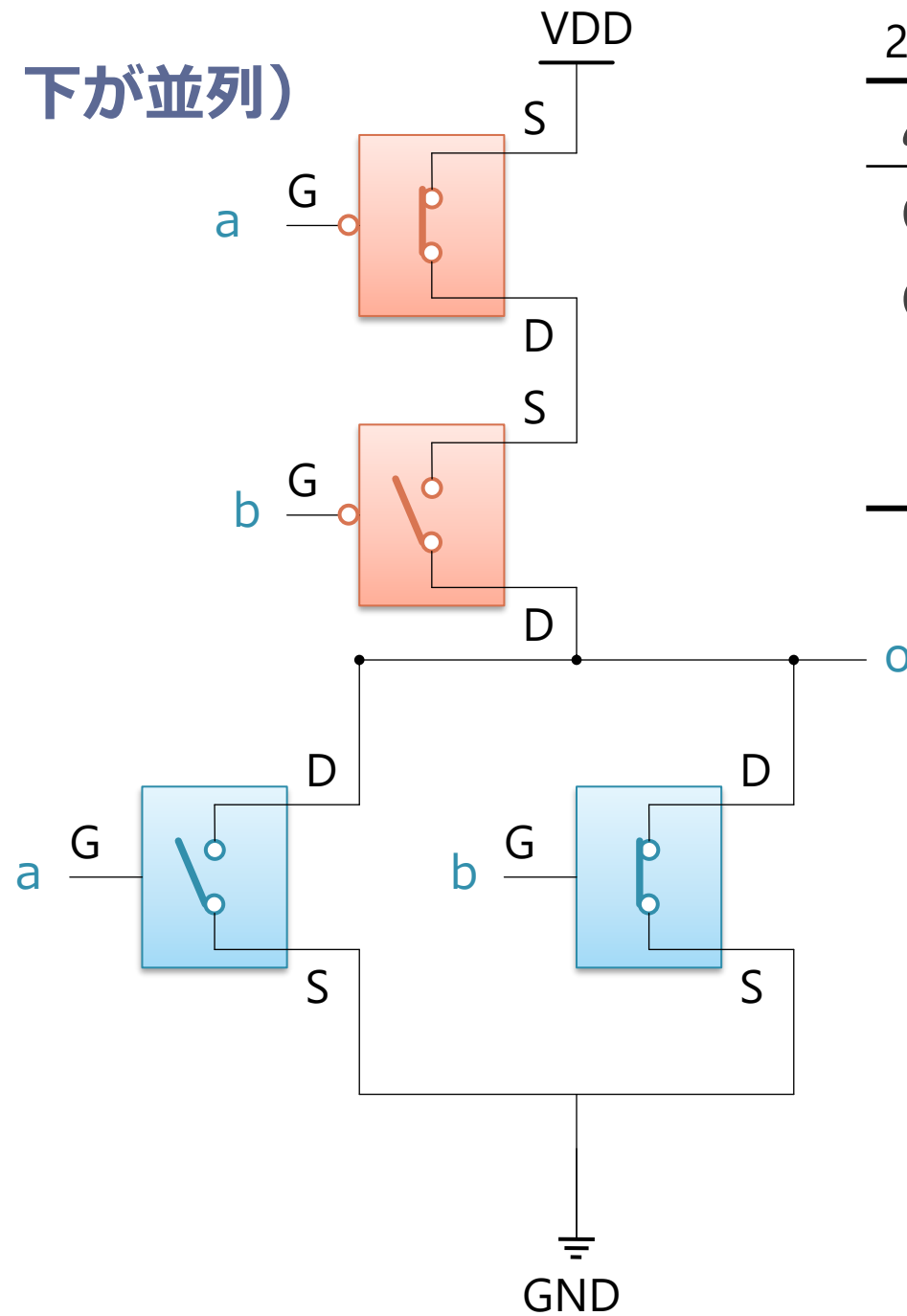
■ OR の作り方

- NOR を作って NOT で反転する
- NAND を作って NOT で反転して AND を作ってたのと同じ

■ 考え方：

- 並列つなぎ：1 つでも ON ならば全体が ON
- 直列つなぎ：全員が ON ならば全体が ON

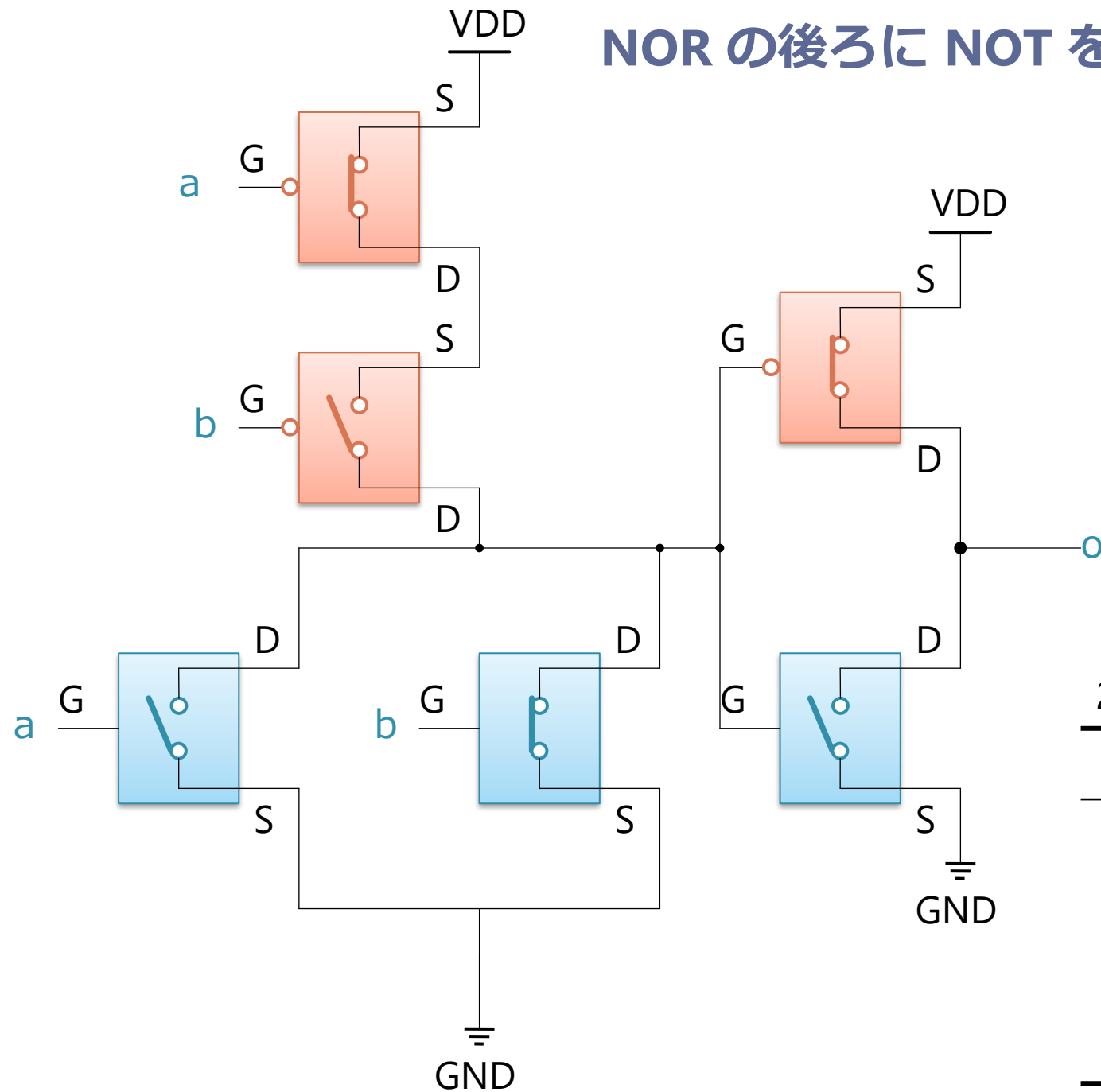
NOR (上が直列, 下が並列)



2入力 NOR の真理値表

a	b	o
0	0	1
0	1	0
1	0	0
1	1	0

NOR の後ろに NOT を接続



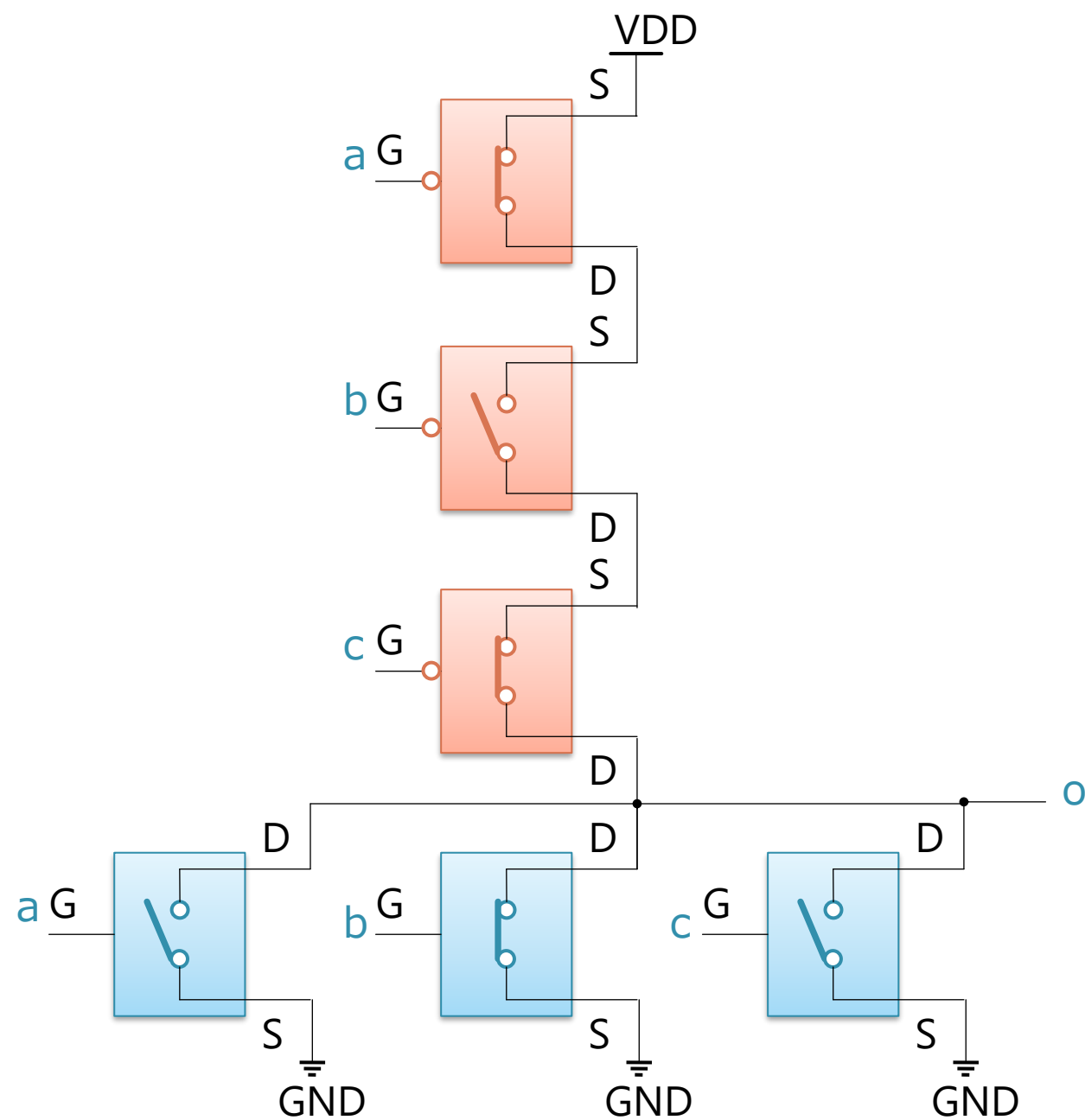
2入力 OR の真理値表

<i>a</i>	<i>b</i>	<i>o</i>
0	0	0
0	1	1
1	0	1
1	1	1

3入力NOR

3入力 NOR の真理値表

<i>a</i>	<i>b</i>	<i>c</i>	<i>o</i>
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

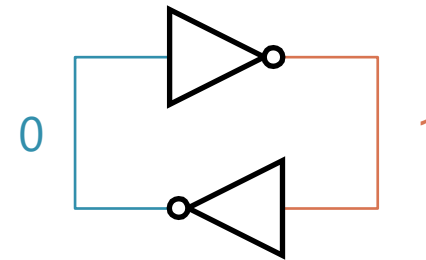
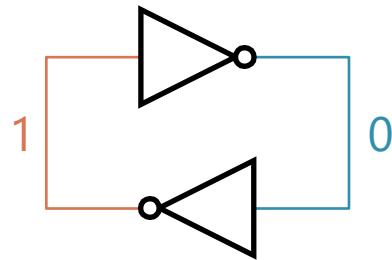


前回の振り返り

記憶素子の原理

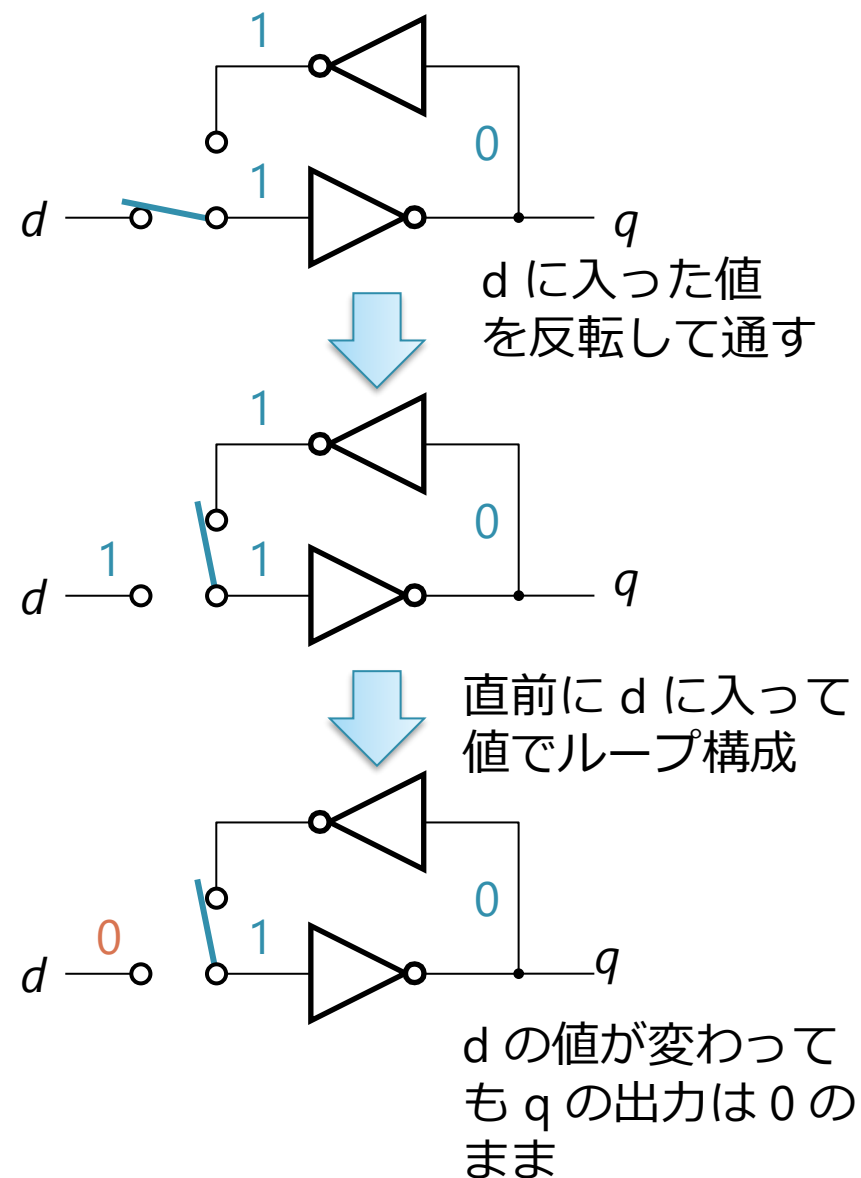
■ 記憶の実現方法：

- 2つの NOT ゲート（インバータ）をループさせた回路により実現
- 以下の2通りの安定状態がある
 - これのどっちになっているによって, 1 bit の情報を記憶



D ラッチの回路

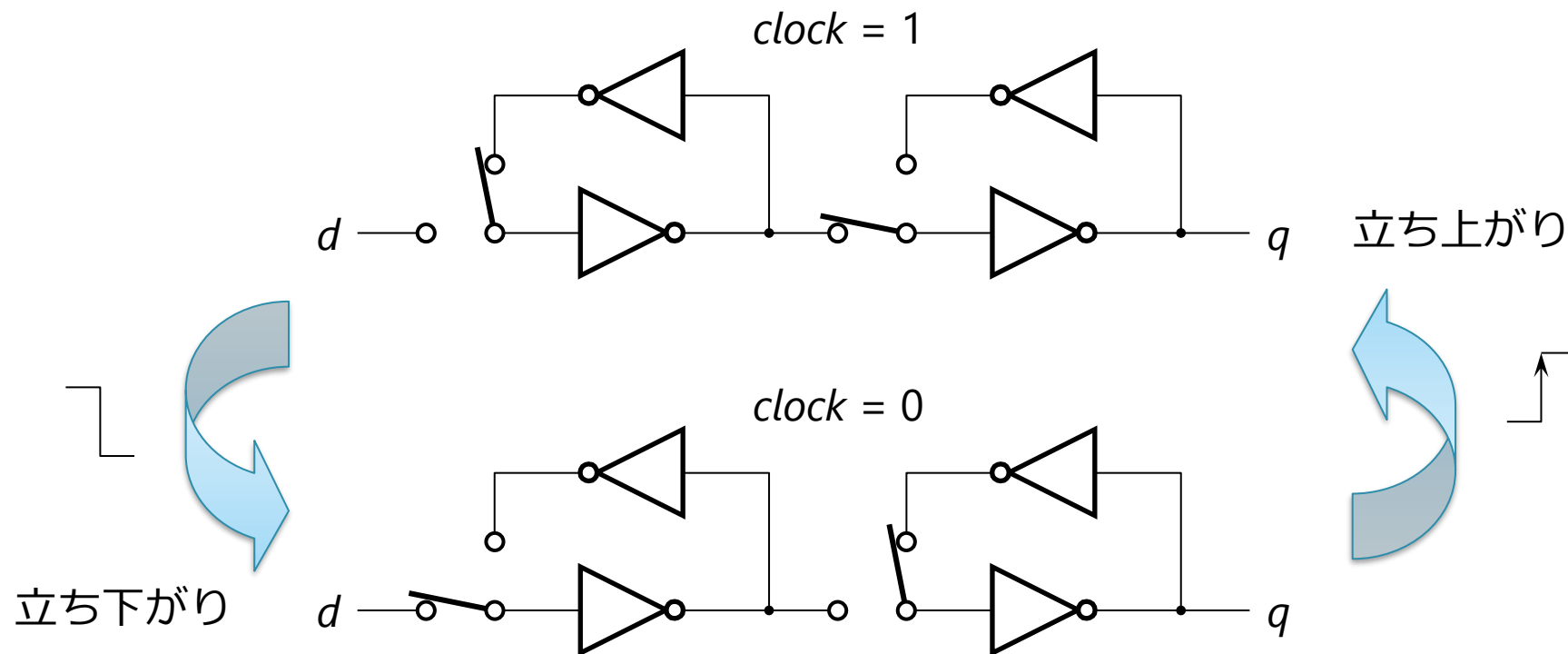
- 構造：マルチプレクサが入ったインバータのループ
 - ◇ ここではマルチプレクサを切り替えスイッチとして説明
 - ◇ クロックの立ち上がりのたびに、スイッチが切り替わる
 - ◇ d の値をループに取り入れ、取り入れた値が q から出力される



D-FF の回路

■ 構造：D ラッチを2つ繋げたもの

- ◇ D ラッチ 1 つだと、半周期は d に入った値が反転して素通しするので使いにくい
- ◇ 2 つ直列に繋げて素通しの期間をなくす

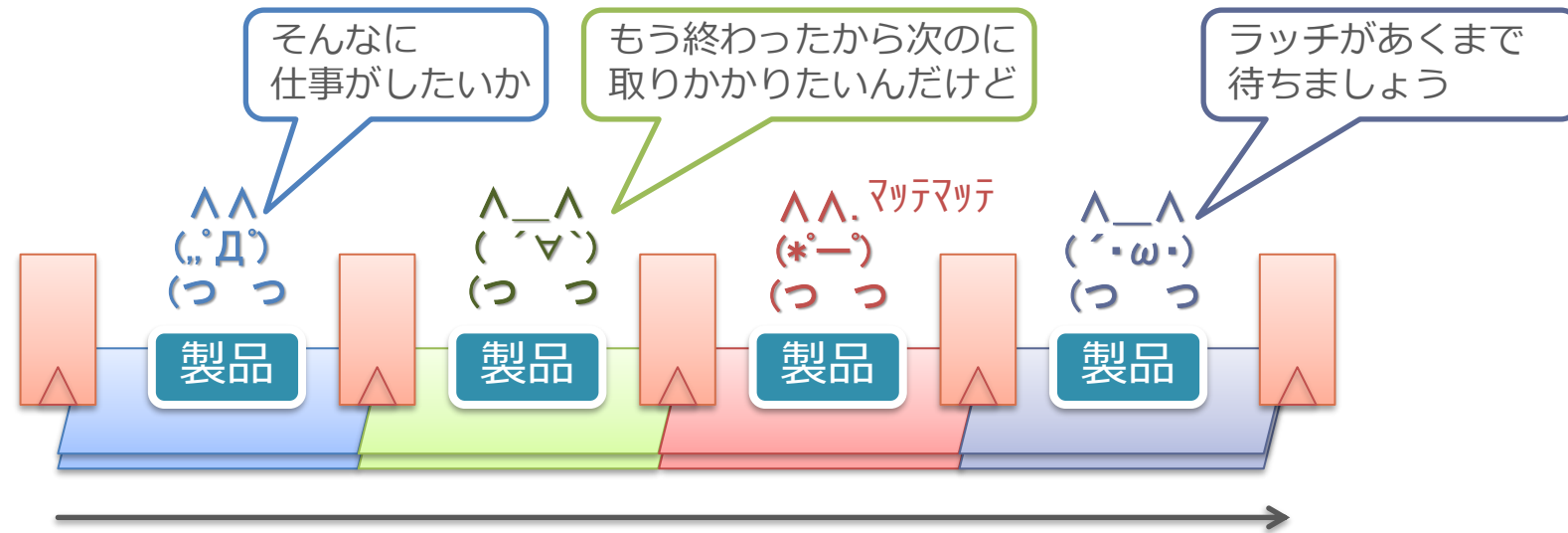


工場のラインを考える（再）



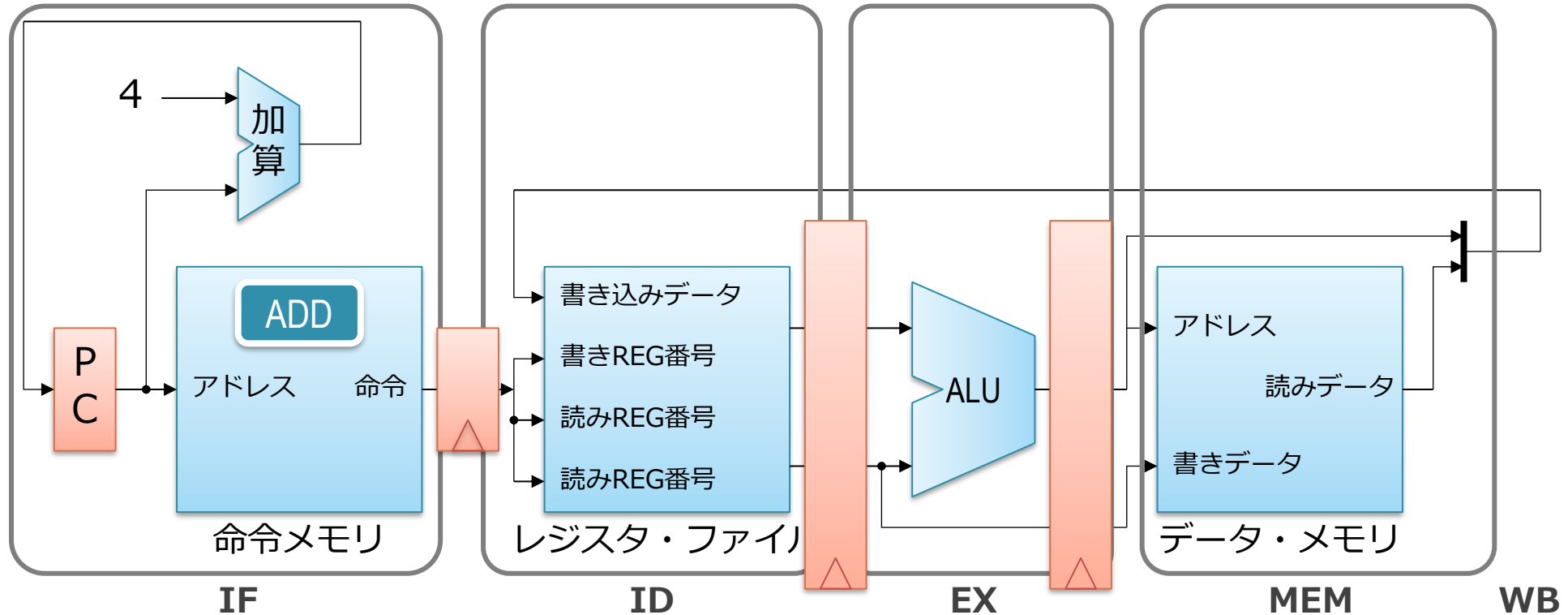
- 実際の工場：複数の製品を同時に流す
 - 各工程を並列して処理することによりスループットを向上
- これが 命令パイプライン

パイプライン・ラッチのイメージ



- 各人の作業が終わっても，ラッチが開くまでは次の人に製品を送れない
 - 複数ステージ間で信号が混じるのを防ぐ
 - 指定された時間までラッチでドアを開かなくするイメージ？
 - 上の絵では，赤い人がまだ仕事が終わっていないので，全体が歩調を合わせるために待っている

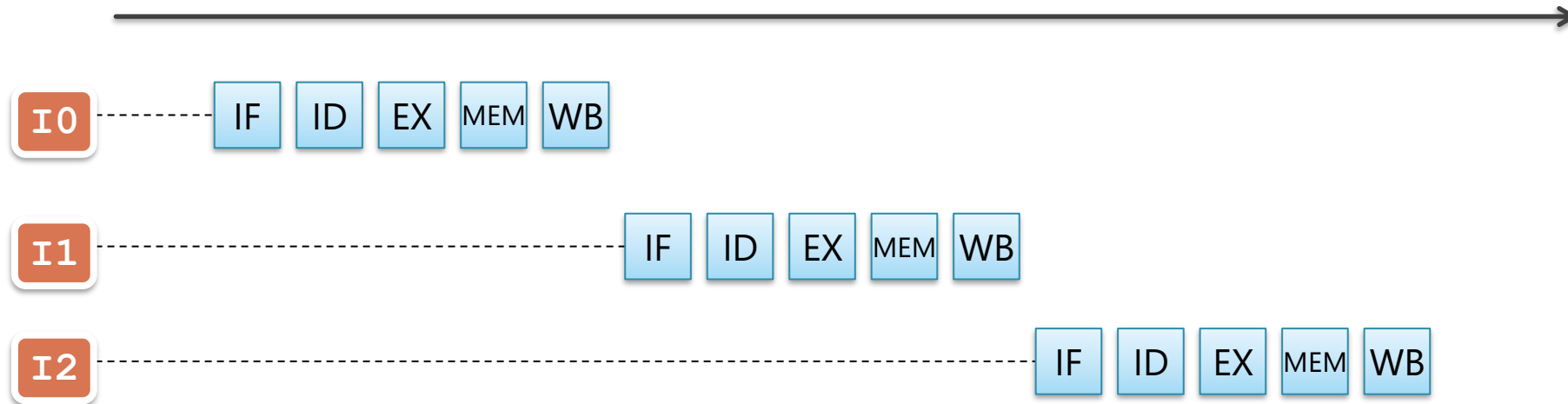
パイプライン化（オーバーラップ）の実現方法



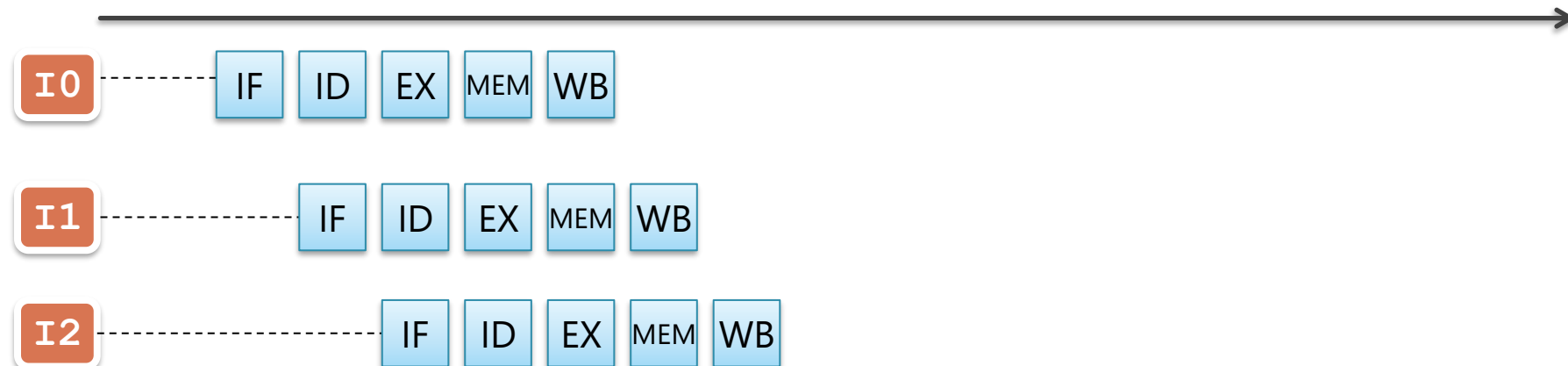
- 各ステージの間に, D-FF (オレンジの四角) を入れる
 - WB の書き込みについては, レジスタ・ファイル自体がクロックに同期して書き込みが行われるので D-FF は不要
- 各ステージの処理が早く終わっても, 次のクロックまでは D-FF で信号の伝搬を止める

パイプライン化による性能（スループット）向上

パイプライン化しない場合



パイプライン化した場合



パイプライン化の意味

- パイプライン化の効果：
 - スループットの向上
 - = 単位時間あたりに処理できる命令の数の増加
 - = 動作クロック周波数の向上
- これらは同じ事を言い換えてるだけ

パイプライン化の限界



- パイプライン段数を増やしていけば、どこまでも速くなるのか？
 - ならない
- 理由：
 1. 回路的な理由による周波数向上の限界（前回の講義で説明）
 2. **アーキテクチャ的な理由（ハザード）による実効性能の限界**

ハザード

ハザード (hazard)

- パイプラインがうまく動作しないこと
 - パタヘネ（教科書）の定義：
「パイプラインにおいて 次のサイクルに次の命令を実行できないこと」
- ハザードの種類：
 - 構造ハザード
 - 非構造ハザード
 - データ・ハザード
 - 制御ハザード

もくじ

1. 構造ハザード：ハード資源の不足に起因

1. 構造ハザードとはなにか？
2. その解決方法

2. 非構造ハザード：バックエッジに由来

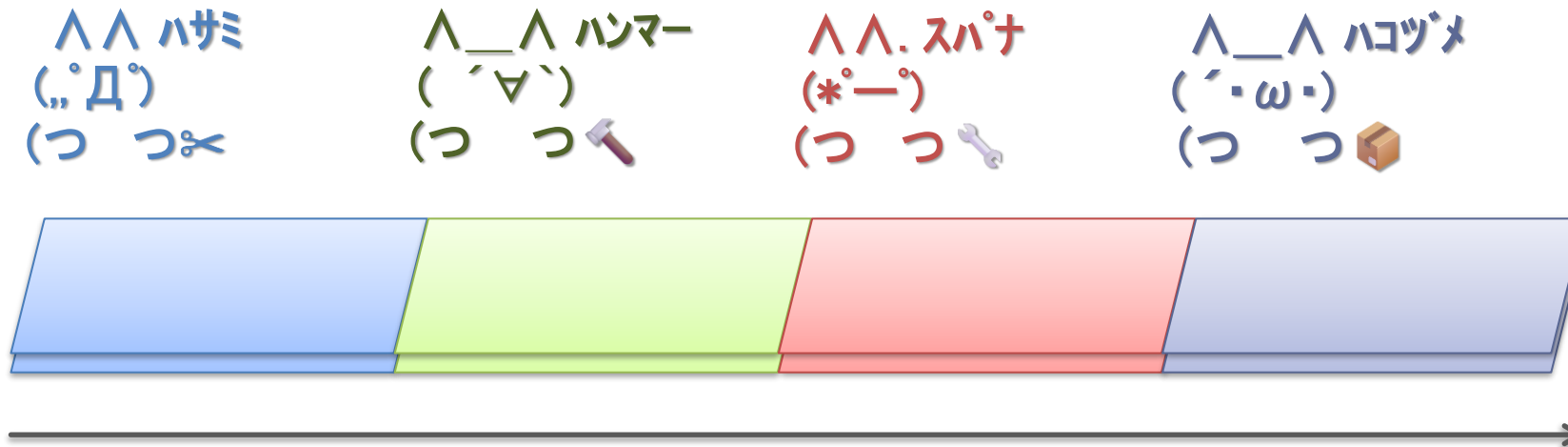
- a. データ・ハザード
- b. 制御ハザード

構造ハザード

構造ハザード

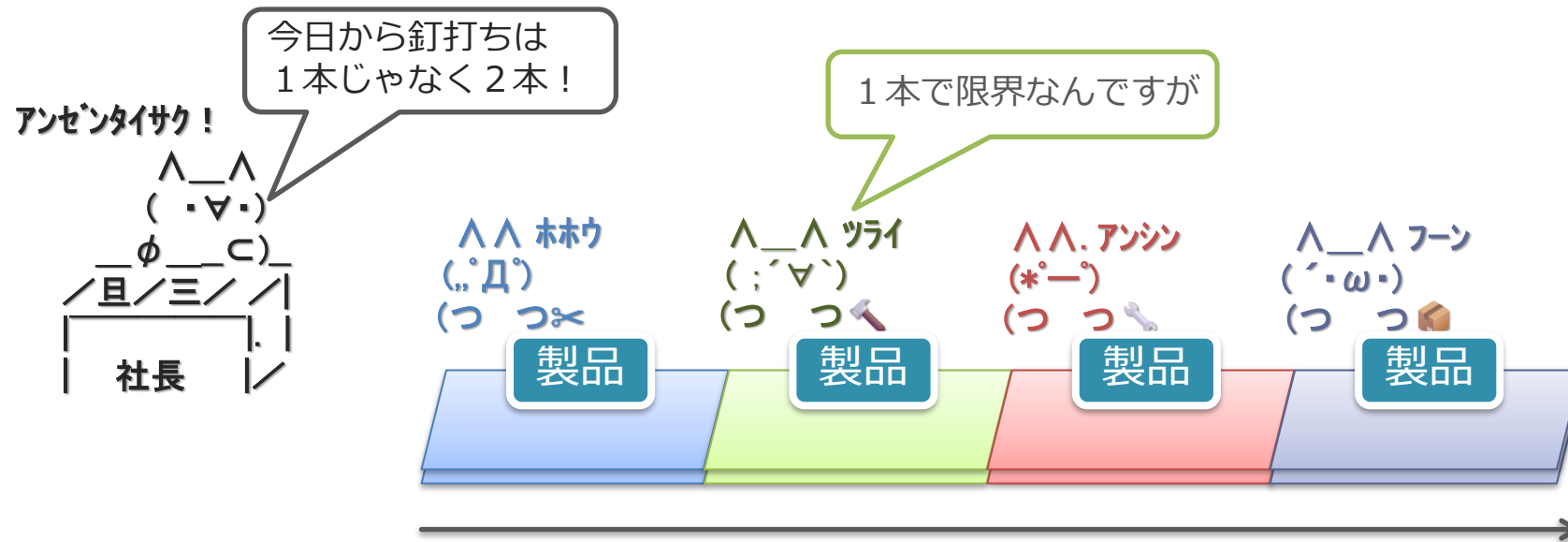
- ハード資源の不足により, パイプラインがうまく動作しないこと
- 工場のラインの例を使って説明

工場のラインをまた考える



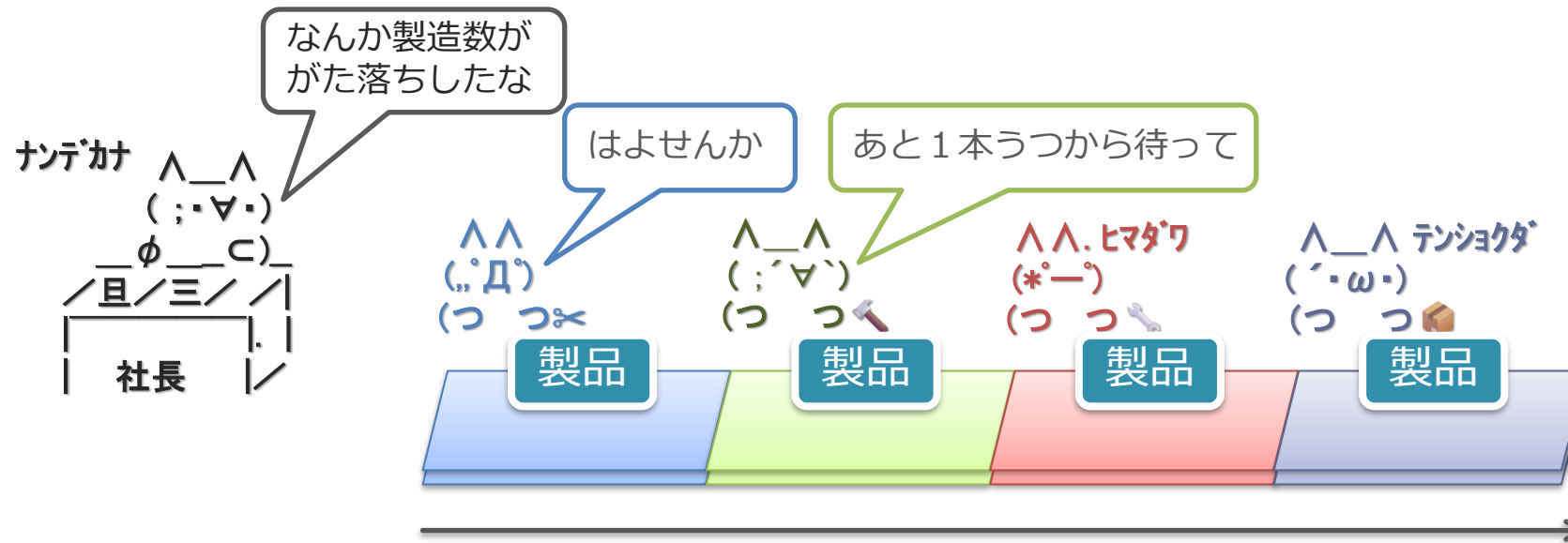
- 各ステージの担当者は製品が流れるまでの間に一定の仕事ができる
 - はさみで紙を 1 枚切る
 - ハンマーで釘を 1 本打つ
 - スパナでねじを 1 個しめる
 - 箱に製品を 1 つ入れる

釘の本数の変更



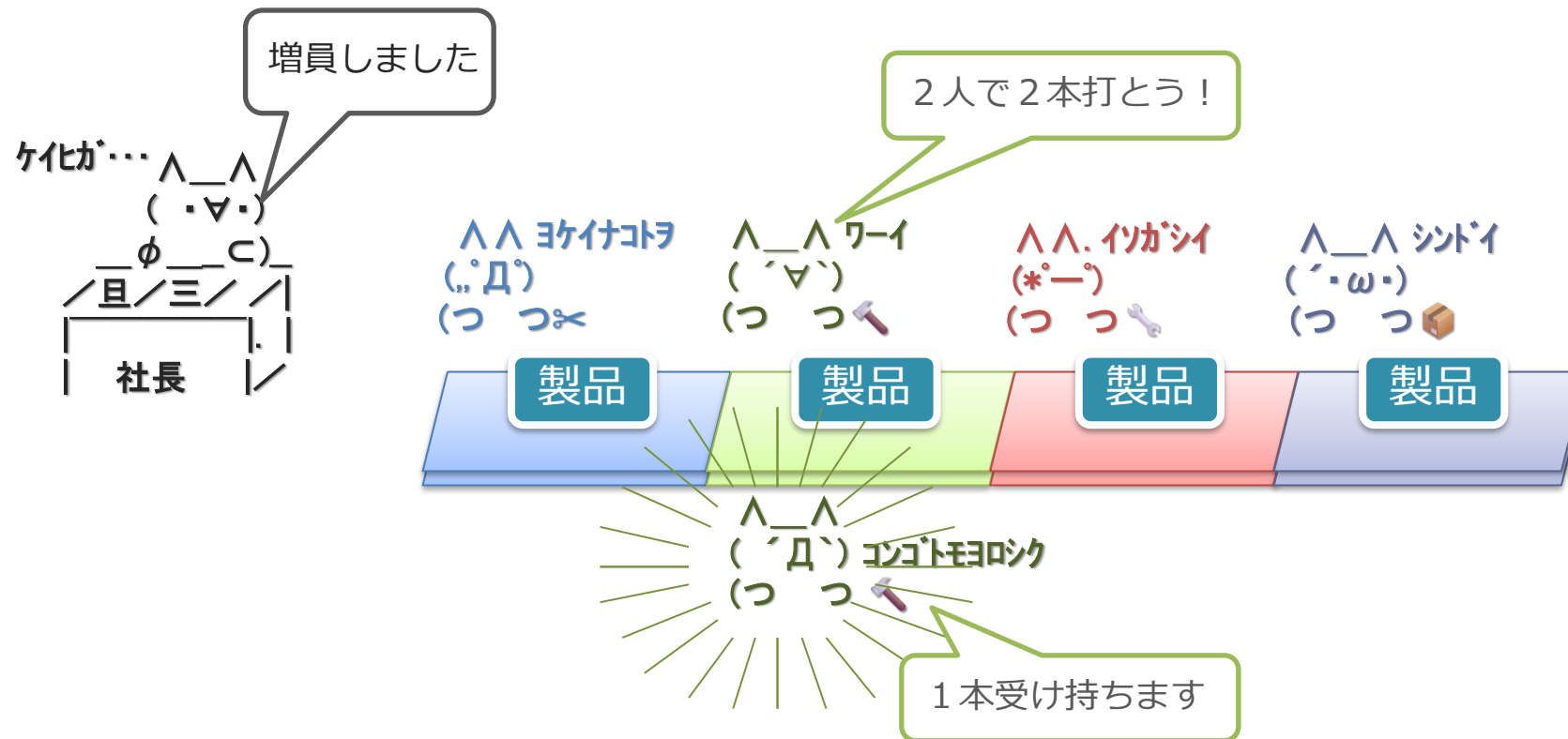
- 製品の安全対策のために釘の本数が1本から2本に
 - しかし、ハンマーを持っている釘打ち担当は単位時間に1本しか打てない

釘打ちの人のところで構造ハザード



- 2人目 (; ' ▽ ') の人は単位時間に1本しか打てない
 - ラインは一番遅い人に合わせて動く（前回の講義より）
 - そこで2本打つまでライン全体が止まる
 - 全体の速度がそこで決まってしまう

増員による構造ハザードの解消



- 1人増やして緑のステージで単位時間に2本の釘が打てるように
 - これがハード資源の追加による構造ハザードの解消
 - ただし、追加しただけ経費がかかる
 - (ハードだとその分複雑になって電力を食う)

コンピュータにおける構造ハザード

■ ハード資源：

- 演算器（FU）, レジスタ・ファイル, メモリ など

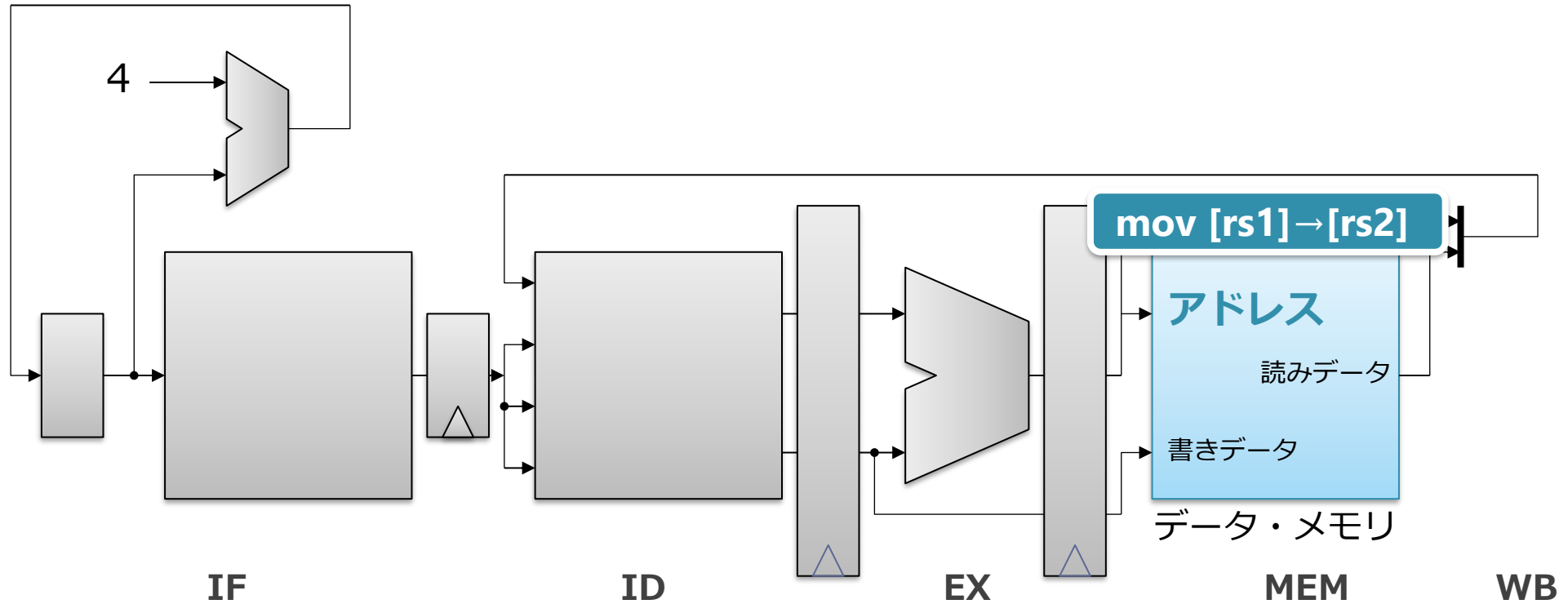
■ 構造ハザード：

- ハード資源の不足により, パイプラインがうまく動作しないこと
- いくつかの例を使った説明, 解消方法について解説

構造ハザードの例 1 : メモリ間 mov

- 例 1 : 仮に `mov [rs1]→[rs2]` のような命令があったとする
 - メモリを読んで、その結果をメモリに書く (=コピー)
- 具体的な動作 :
 - `rs1` で指定されるアドレスのメモリの値を読んで、
 - `rs2` で指定されるアドレスのメモリに書き込む
- 実際に、x86 にはこのような命令がある

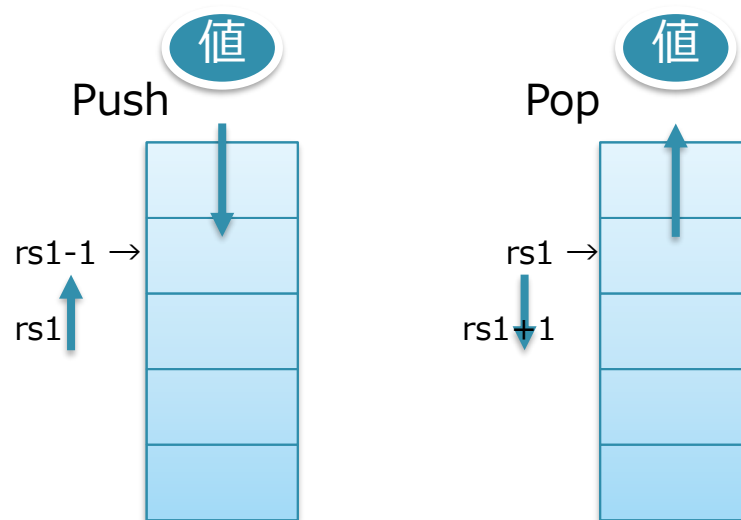
mov [rs1]→[rs2] // [rs1]→[rs2] へのコピー



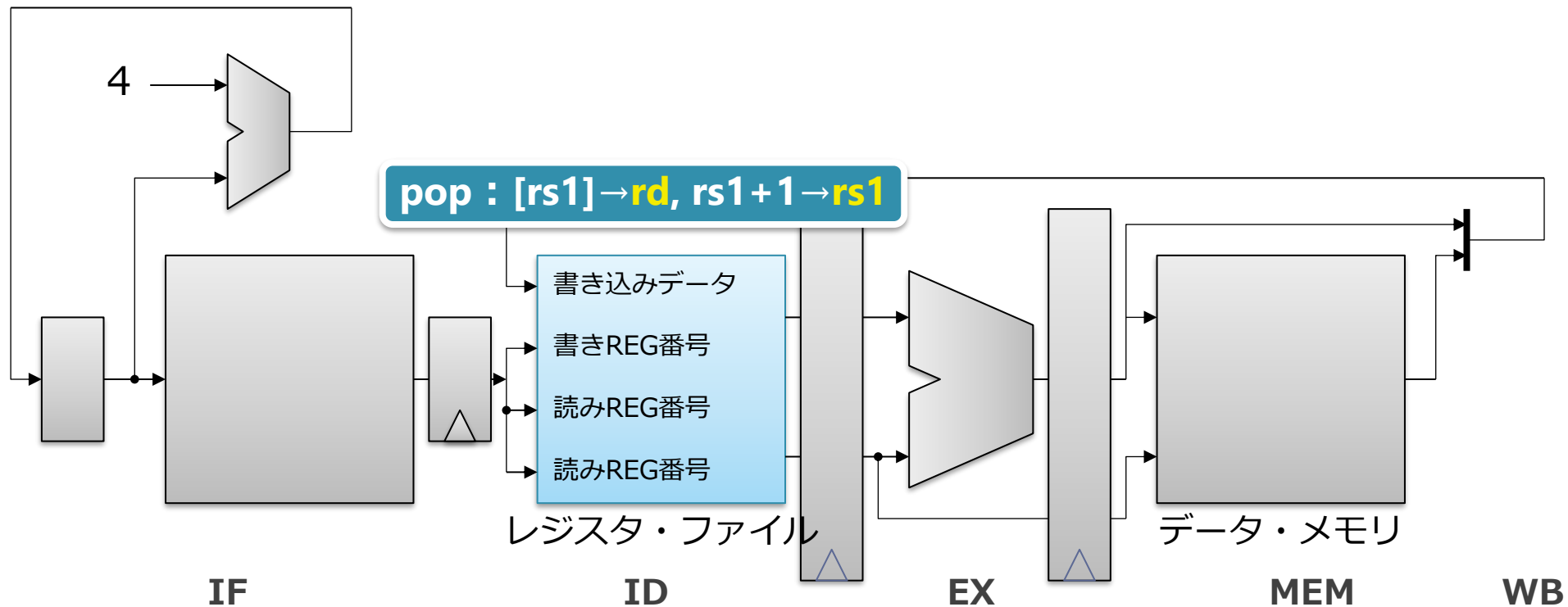
- ある1つのサイクルにメモリを「読んで」「書く」必要がある
 - しかし, データ・メモリのアドレスの口は1つしかない
 - MEM ステージでデータ・メモリの読みと書きが同時にできない

構造ハザードの例 2 : push/pop

- x86 や ARM ではスタック操作のための push/pop 命令がある
 - push : $rs1-1 \rightarrow rd$, $r2 \rightarrow [rd]$
 1. スタック・ポインタ（が入ってるレジスタ）をデクリメントし,
 2. それをアドレスにしてメモリに値を書き込む
 - pop : $[rs1] \rightarrow rd$, $rs1+1 \rightarrow rs1$
 1. スタック・ポインタをアドレスにして値を読む
 2. スタック・ポインタをインクリメント



pop : [rs1]→rd, rs1+1→rs1



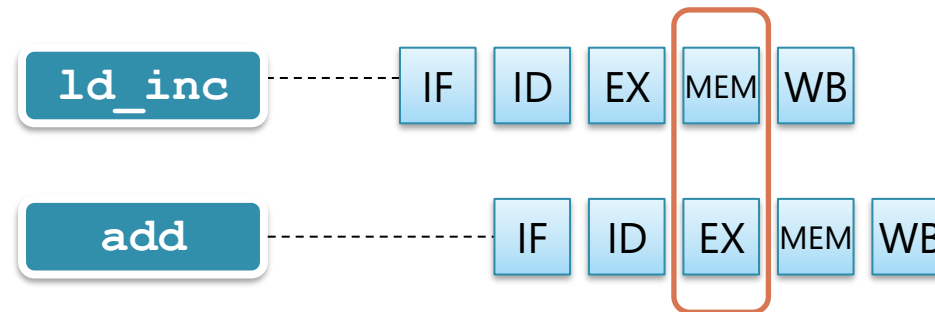
- WB ステージでレジスタに `rd` と `rs1` の2つを書き込む必要がある
 - 2つのレジスタが1つの命令により更新されている
 - レジスタ・ファイルへの書き込みは、同時に2つはできない

構造ハザードの例 3

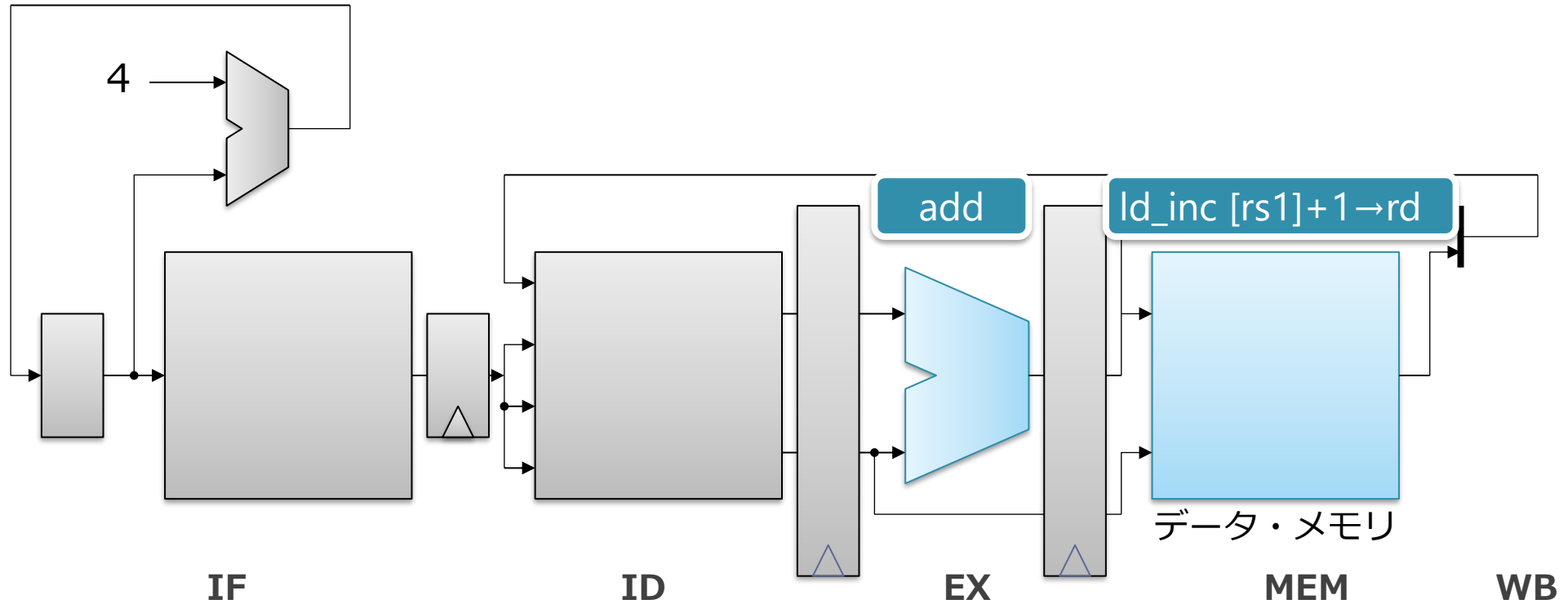
- 使用資源の異なるステージ間のぶつかりでも起きる
 - これまでの例は, 同じステージ内で資源が足りない例
- `ld_inc [rs1]+1→rd` のような命令があったとする
 1. `rs1` の指すアドレスからメモリを読む
 2. 読んだ値にさらに + 1 してから `rd` に書く
- 一見, 資源は足りているようだが…
 - レジスタの読み書きは 1 つずつしかない
 - メモリも 1 カ所を読むだけ
 - 加算も 1 回行うだけ

構造ハザードの例 3

- `ld_inc [rs1]+1→rd` と `add` が連続した場合：
 - `ld_inc` で, MEM ステージから読んだ値を加算しようとしても,
 - そのサイクルは後続の `add` が演算器を使っているので使用できない



ld_inc [rs1]+1→rd と add が連続した場合



- 命令は EX ステージ以外では、演算器にはアクセスできない
 - そうしないと、他の命令の EX ステージとかち合ってしまう

構造ハザードの解決方法

- 解決方法

1. ハードウェアの増強
2. 時分割処理

解決方法 1 : ハードウェアの増強

■ ハードウェアを増強する

- `mov [rs1]→[rs2]`
 - 複数箇所のメモリを同時に読み書きできるように
- `pop`
 - レジスタに2つ同時に書き込めるように
- `ld_inc [rs1]+1→rd`
 - MEM ステージに専用の加算器を追加

解決方法 1 : ハードウェアの増強

- 利点 : オーバーヘッドをいとわなければ, 基本これで解決
- 欠点 : 回路規模が増える
 - 1. 機能の増強量に比例した回路が必要
 - なにも考えないで対応していくと, ものすごい数の回路になる
 - 例 : ARM は全 16 レジスタを一気にメモリに書ける命令がある
 - 2. 機能の増強量に対して, 線形より大きなオーダーで回路規模が増える場合もある
 - 加算器などなら, 増やした数の分だけ線形に回路が増える
 - メモリやレジスタは, 同時に読み書きできる数の2乗で回路が大きくなる性質がある

構造ハザードの解決方法

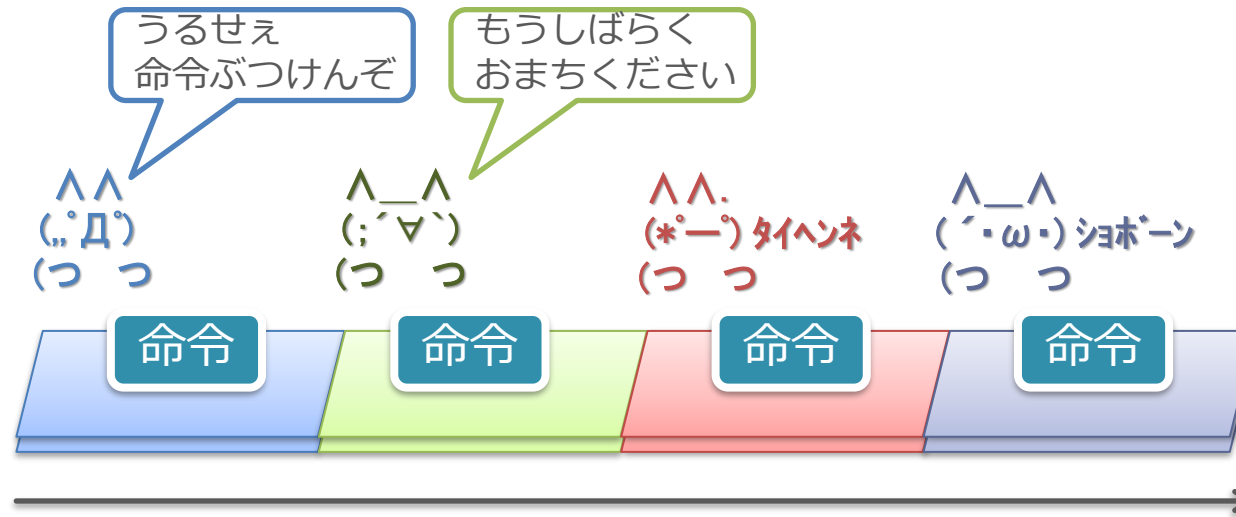
■ 解決方法

1. ハードウェアの増強
2. **時分割処理**

解決方法 2 : 時分割で処理

- 構造ハザードの原因：
 - ハードウェア（の機能）が足りない
- **パイプラインを止めて**, 複数のサイクルをかけて処理する
 - `mov [rs1]→[rs2]`
 - メモリを読んだあと, 次のサイクルで書きこむ
 - `pop`
 - 1 つレジスタに書いたあと, 次のサイクルで書き込む
 - `ld_inc [rs1]+1→rd`
 - `ld_inc` が MEM で値を読んだら, 次のサイクルで +1

なぜパイプラインを止めるのか

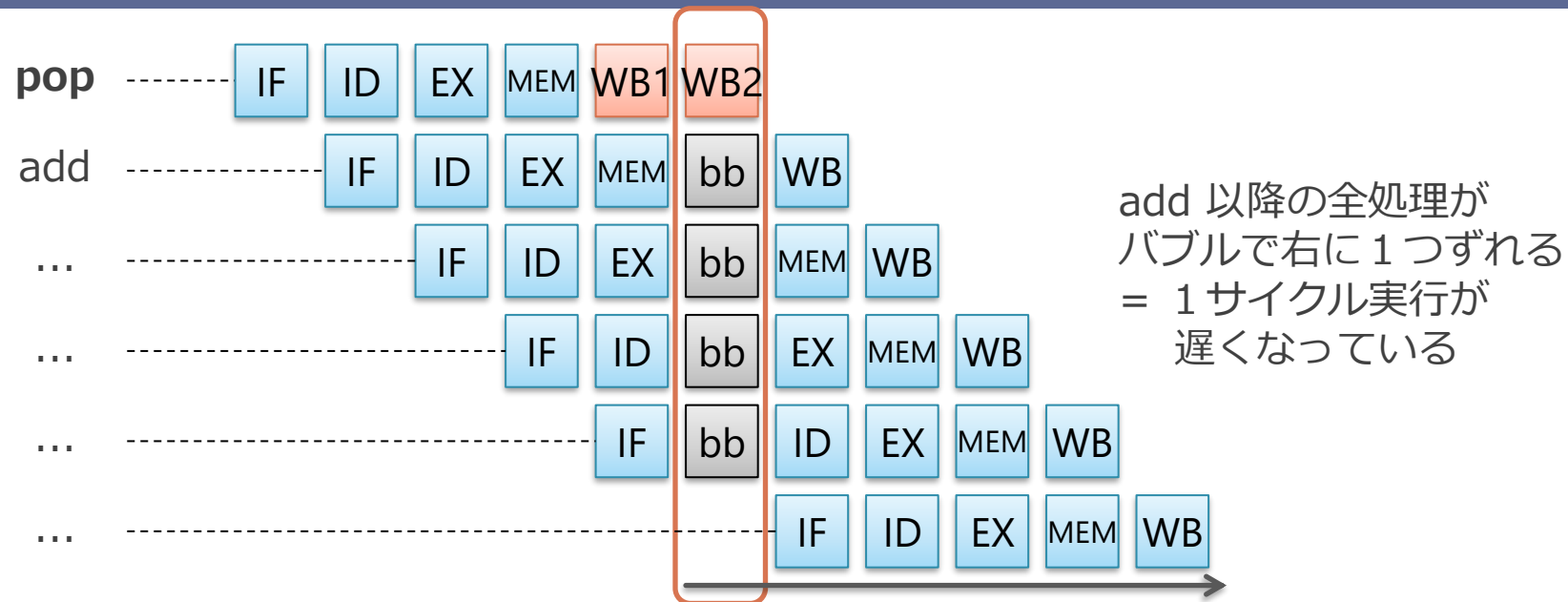


- 上流を止めないと破綻する
 - $(; \cdot \nabla \cdot)$ が複数サイクルをかけて仕事をしている場合、命令はそこにとどまり続ける
 - その間は上流をとめないと命令をおく場所がないし、依存関係がまもられない
- $(* \cdot -)$ より下流は流れていっても、この場合は問題ない

パイプラインを止めること

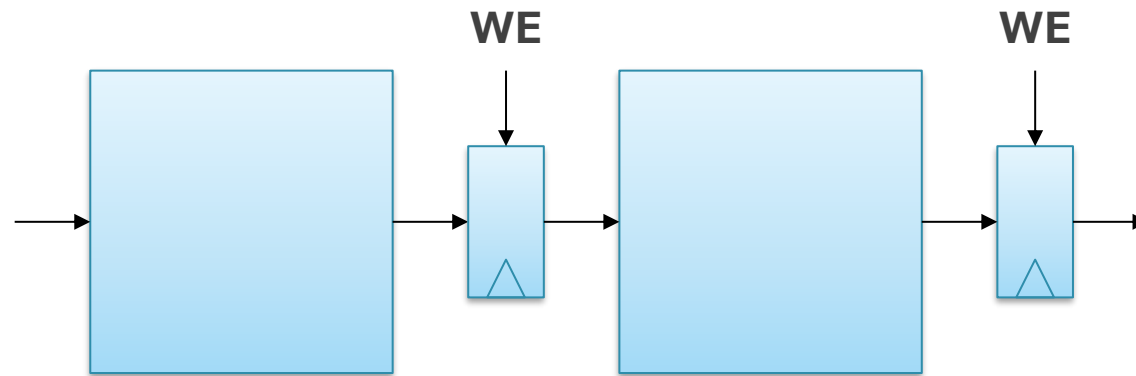
- パイプラインを止めるとことを「ストール」や「インターロック」という
 - 本や人によって、意味や使い方が微妙に統一されていない
 - この講義では、以降はストールで統一

ストールの動作



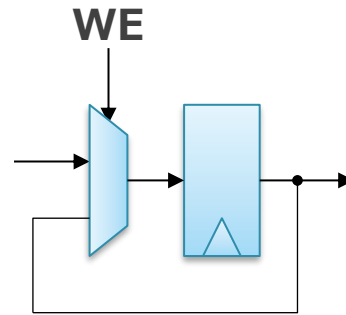
- `pop` : 1つレジスタに書いたあと、次のサイクルで書き込む
 - WB1 と WB2 の2サイクルで書き込む
 - WB2の間は上流を全て止める
- パイプライン・チャート上では上記のようになる
 - 止める原因の命令の下が全部右にずれる
 - ずれた部分の空き (bb) を「バブル (bubble)」とよぶ

ストールの実現方法



- 回路的には、Write Enable (WE) つきの D-FF を使う
 - WE が 0 のサイクルは書き込みが行われない
 - ストールさせたい時は、そのステージの WE を 0 に

WE つき D-FF の実現方法



- たとえば D-FF とマルチプレクサで作れる
 - WE が 1 の時は, 左からきた入力を選んで書き込む
 - WE が 0 の時は, その時の自分自身の出力を選んで書き込む

非構造ハザード

もくじ

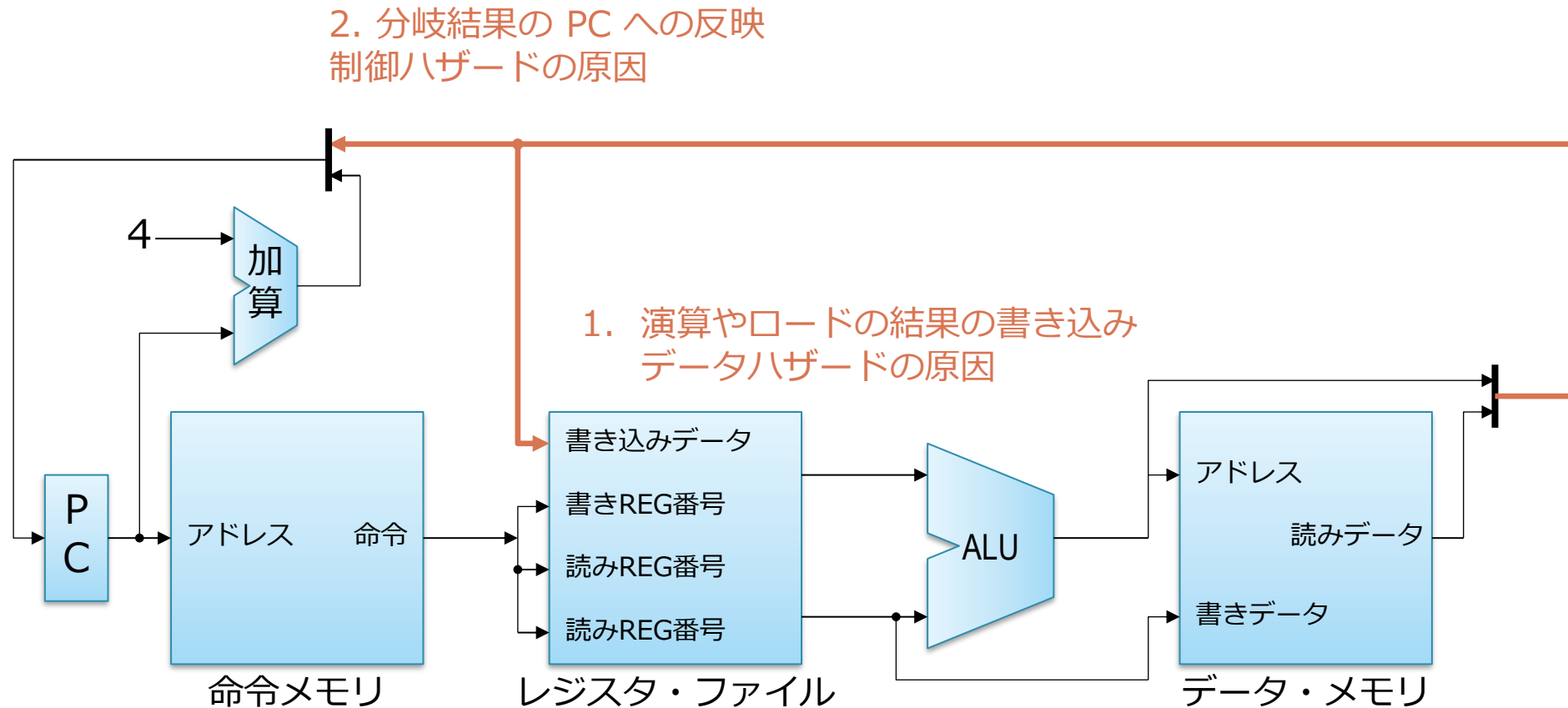
1. 構造ハザード：ハード資源の不足に起因

1. 構造ハザードとはなにか？
2. その解決方法

2. 非構造ハザード：バックエッジに由来

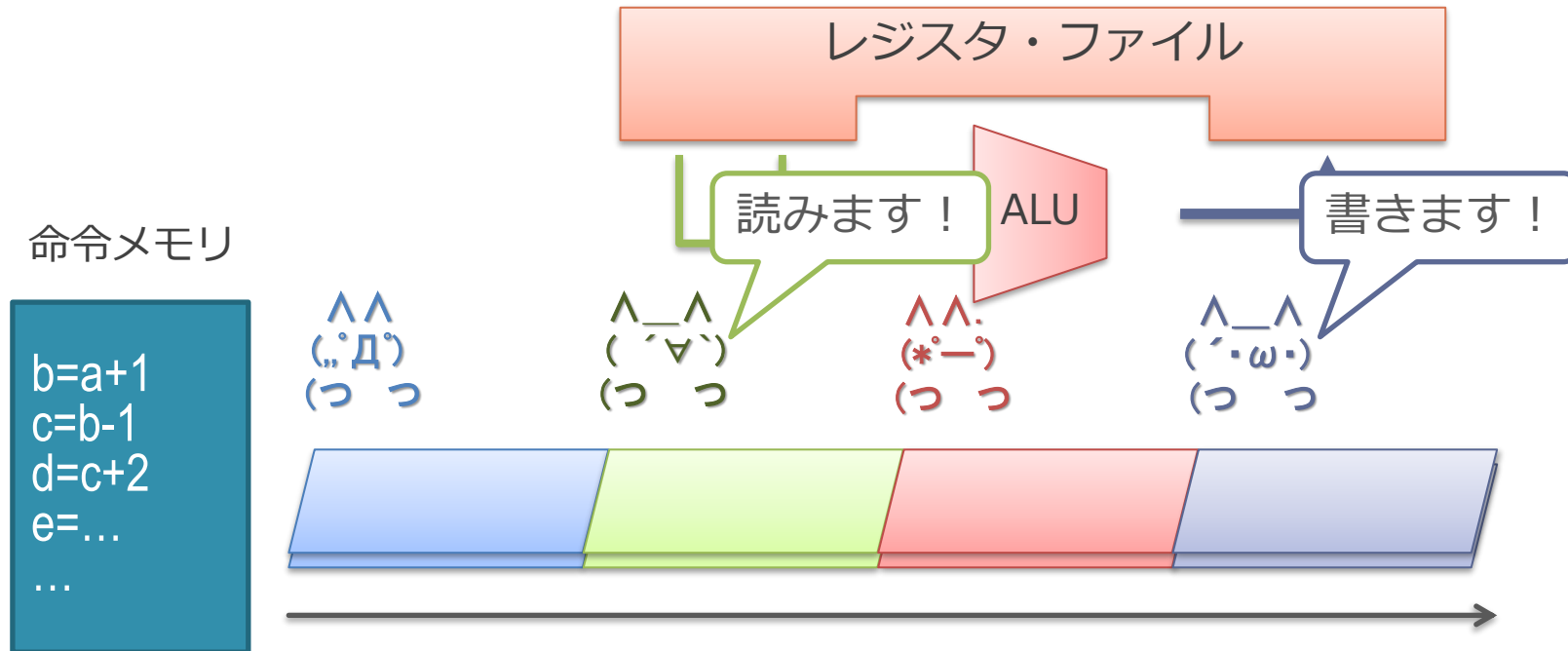
- a. データ・ハザード
- b. 制御ハザード

バックエッジとは：逆方向（右から左）にいく信号



- バックエッジがあるため、命令を単純に流せない場合がある
 - 工場のラインのように、一方向に流せない

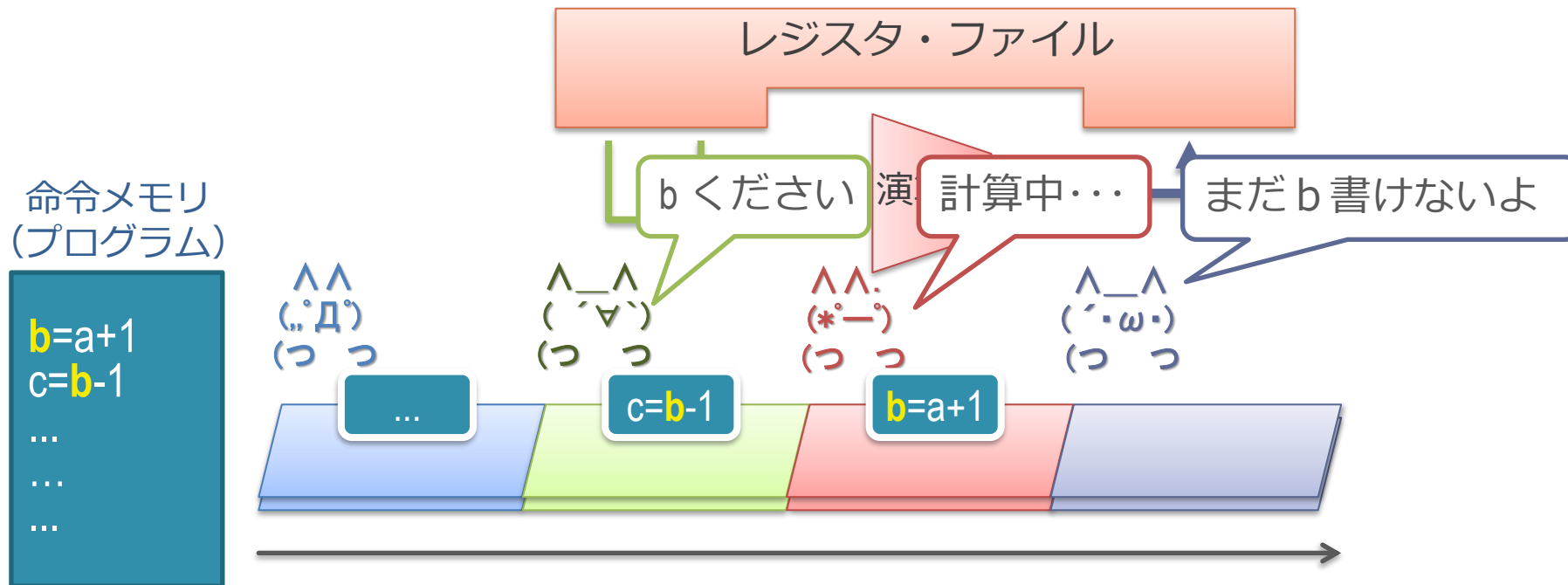
データ・ハザード



■ レジスタ・ファイルへのアクセス

- 演算の入力は(`▽`)の人がレジスタ・ファイルから読み出す
- 演算の結果は(`ω`)の人がレジスタ・ファイルに書き込む

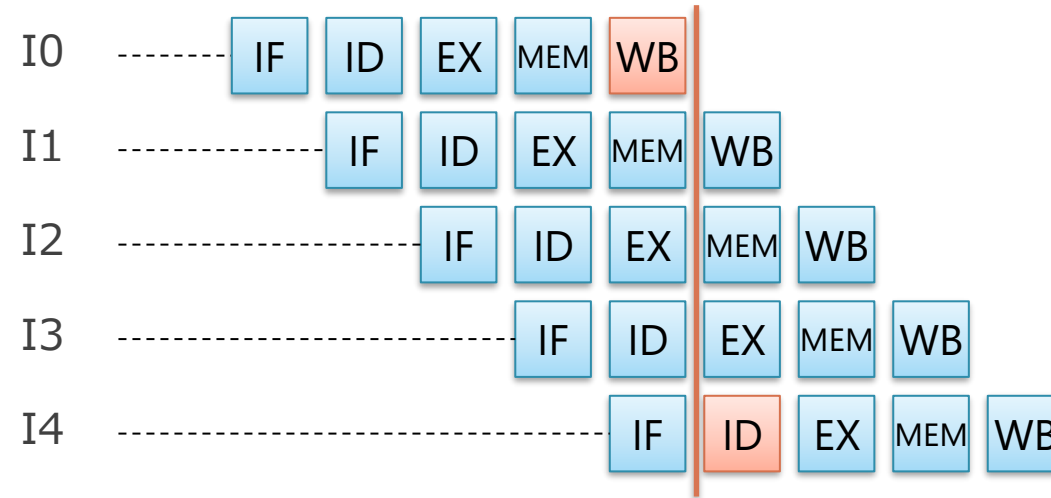
データ・ハザード



■ 直前の命令の結果を使う命令が現れた場合：

- (` } \nabla \text{ `) の人が $b=a+1$ の結果を読もうとしても,
- $(\text{ * } \text{ ` } \text{ -- } \text{ ` })$ の人がまだ計算中でレジスタ・ファイルに b が書けていない
- (` } \cdot \omega \cdot \text{ `) の人が計算結果をかけるのはさらに次のサイクル

データ・ハザード



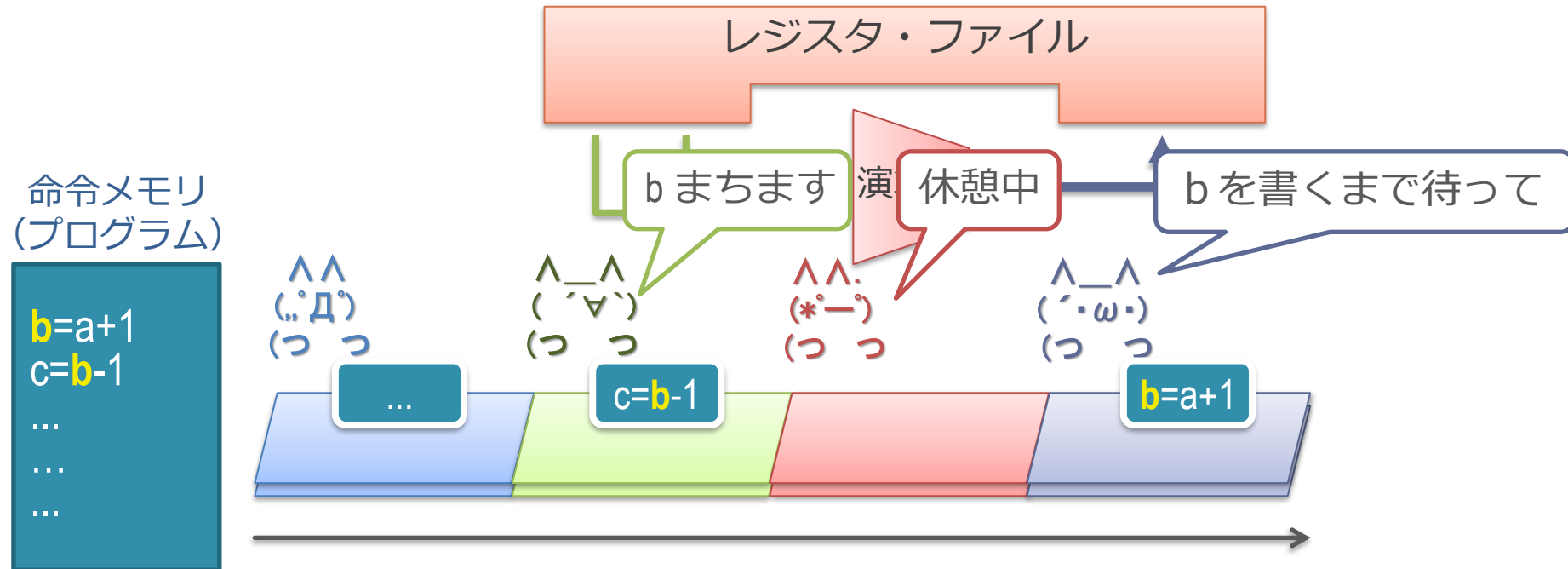
- I0 の WB が終わるまで、その結果はレジスタに書き込まれない
 - I4 までは、その値がレジスタから得られない
 - ID ステージでレジスタを読むため

データ・ハザードの解消方法

■ 解消方法

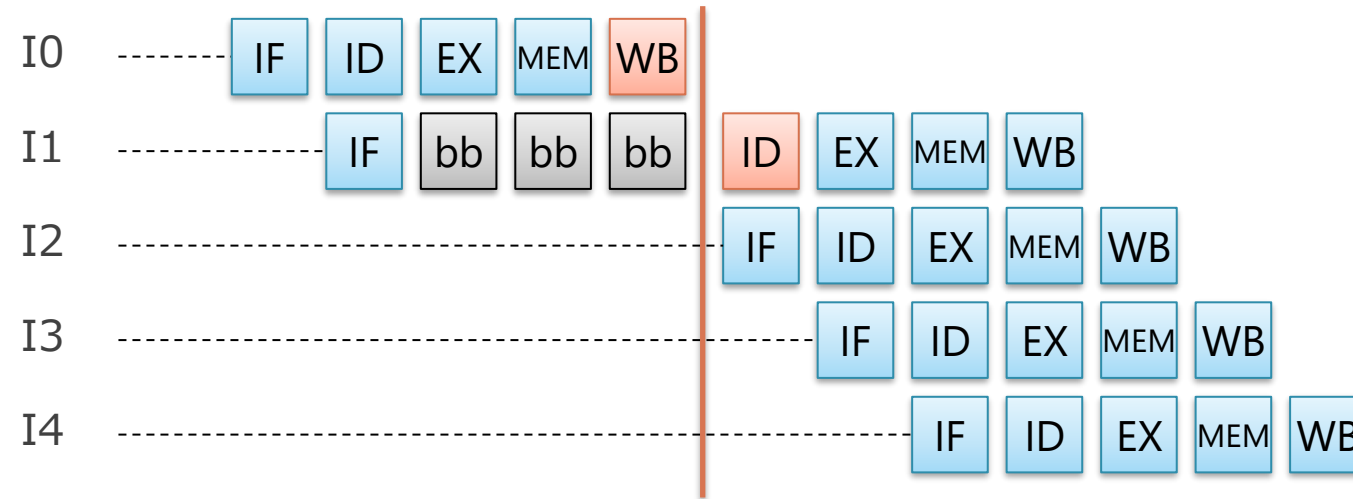
1. ストールさせる
2. 遅延スロット（なにもしない）
3. フォワーディング

1. ストール



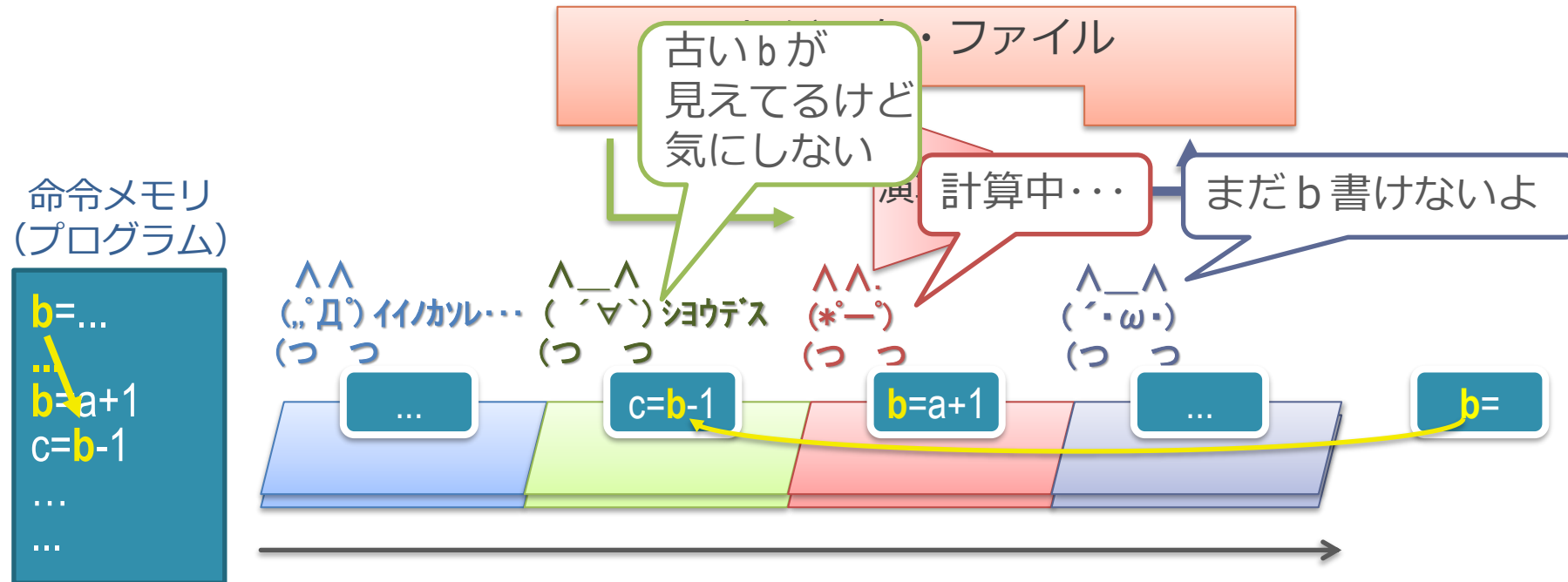
- 直前の命令の結果を使う命令が現れた場合：
 - (´・ω・) の人が計算結果をレジスタに書くまで, (´▽`) の人と上流のラインを止める
 - 間にバブル (何もしないステージ) が入る

1. ストールさせる



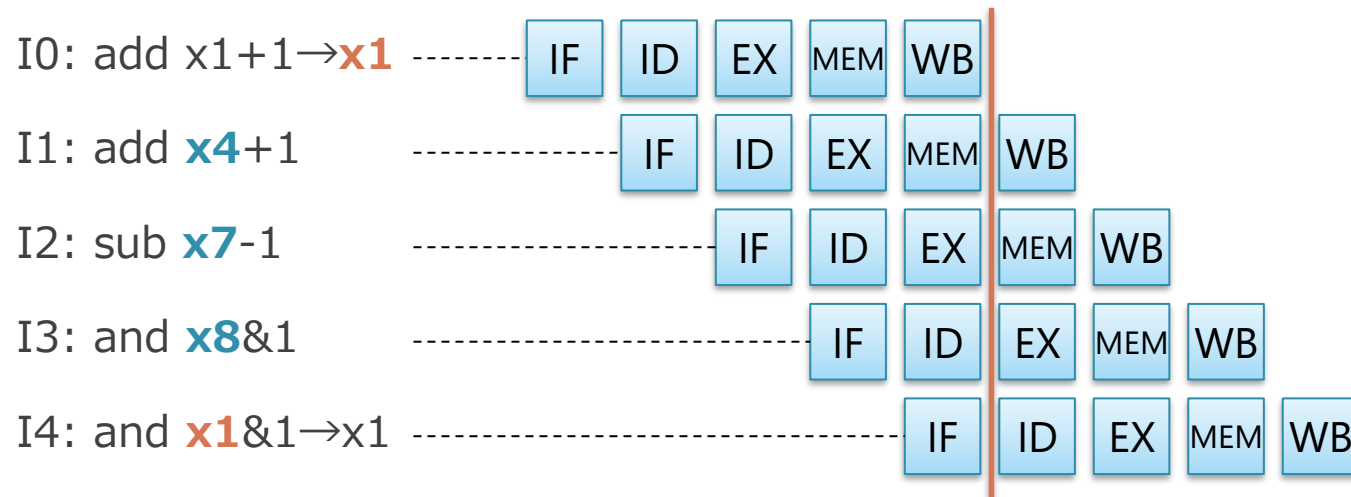
- I0 の WB が終わるまで、後続の命令を遅らせる
 - I1 の ID が, I0 の WB の右にくるまでストール
 - I1 は I0 の結果を使える
- 欠点 : プログラムの実行がとても遅くなる

2. 遅延スロット (なにもしない)



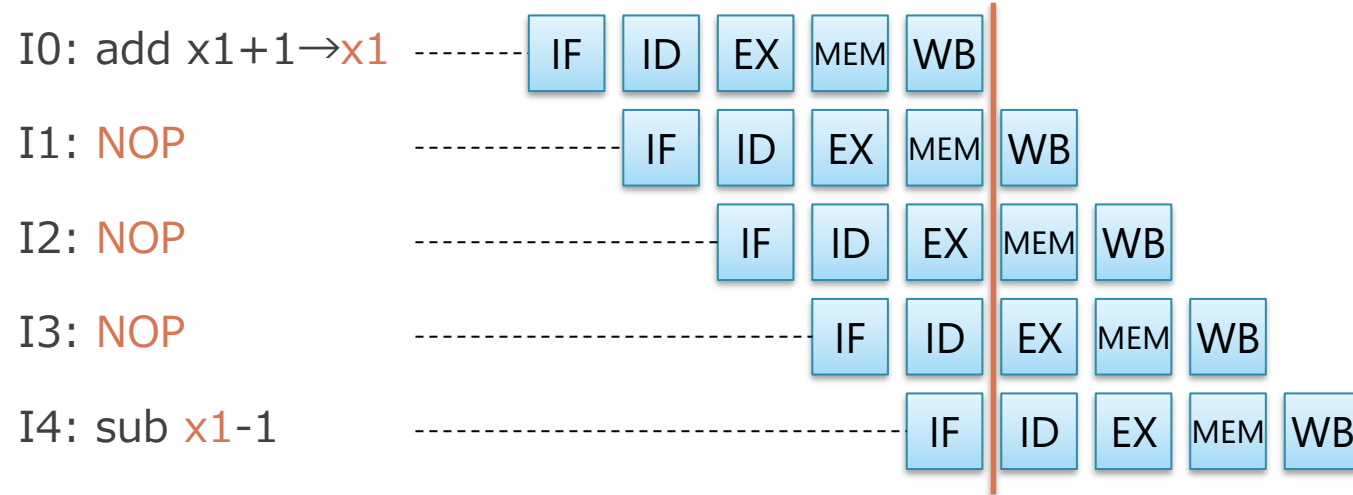
- 直前の命令の結果を使う命令が現れた場合：
 - (' ▽ ') の人は, その時レジスタ・ファイルにある b を気にせず読んでしまう
 - 2つ前の命令の結果の b が読めてしまう
 - そういう仕様ということにする

2. 遅延スロット（なにもしない）



- ここに I0 の結果を使わない命令を入れれば、性能低下はない
 - この直前の命令の結果が見えない部分を「遅延スロット」と呼ぶ
 - この図では、遅延スロットが 3 命令分ある
 - それらは I1, I2, I3 は I0 の結果を使っていないので問題無い
 - 遅延スロットへの命令挿入はコンパイラががんばる
 - 人力でアセンブリ言語でがんばることもある

NOP の挿入



- もしそのような命令がない場合,
 - NOP (No Operation) と呼ぶ何もしない命令をいれる
 - これもコンパイル時にいれておく必要がある
- 上の例は, x1 に 1 を足した結果を使う以外の処理がなかった場合

遅延スロットの利点

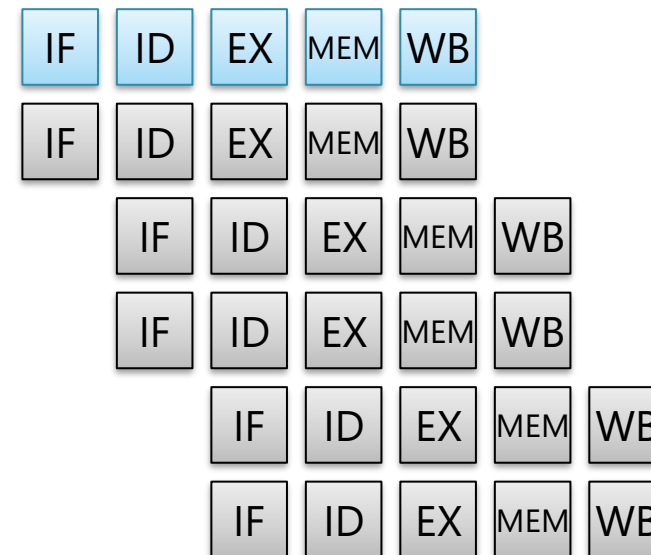
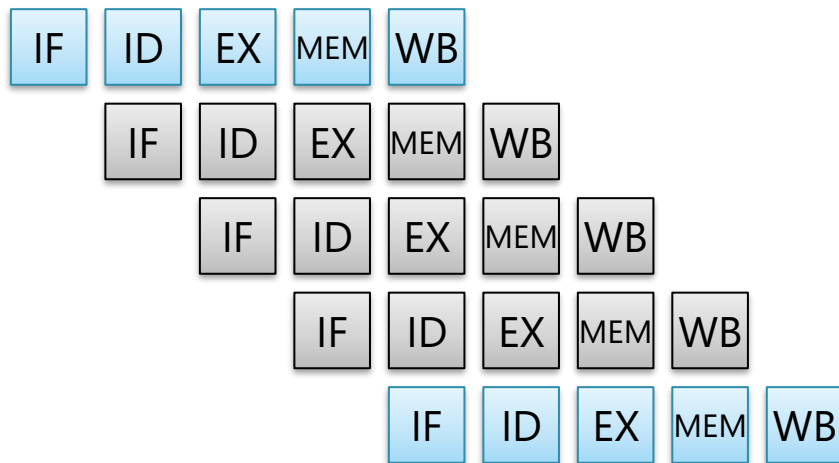
- 利点：

- なにもしないので，ハードは最も単純
- 並列にできる命令があれば，性能も下がらない

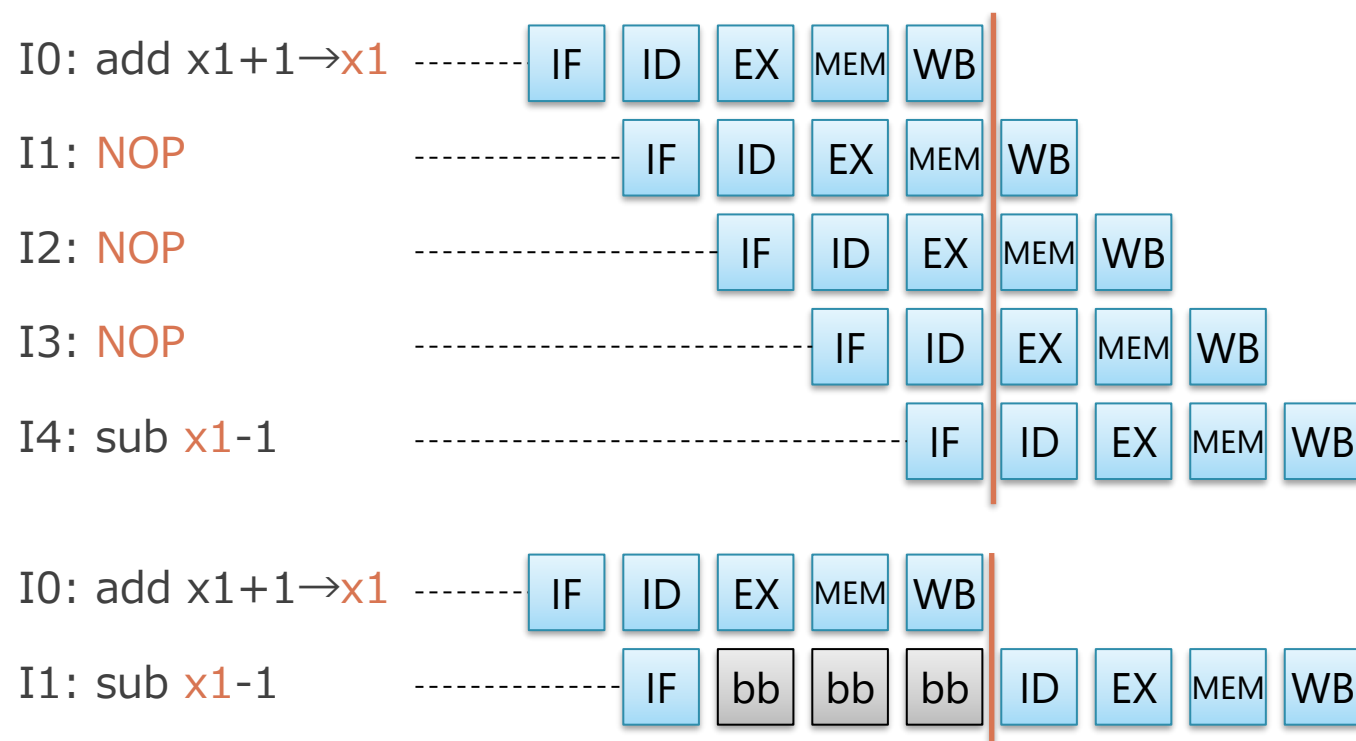
遅延スロットの欠点

- 欠点：「仕様」なので、一度決めると変えられない
 - 後からパイプラインの段数や構造を変えると互換性がなくなる
 - クロックをあげるために、段数を増やせない
 - 複数の命令を同時処理しようとしたときにも互換性がなくなる
 - MIPS では遅延スロットが 1 命令分、仕様として存在
 - 互換性のためにこれを忠実に再現するため後年は逆に複雑化

2 命令同時処理すると、遅延スロットが増える



遅延スロットの欠点2



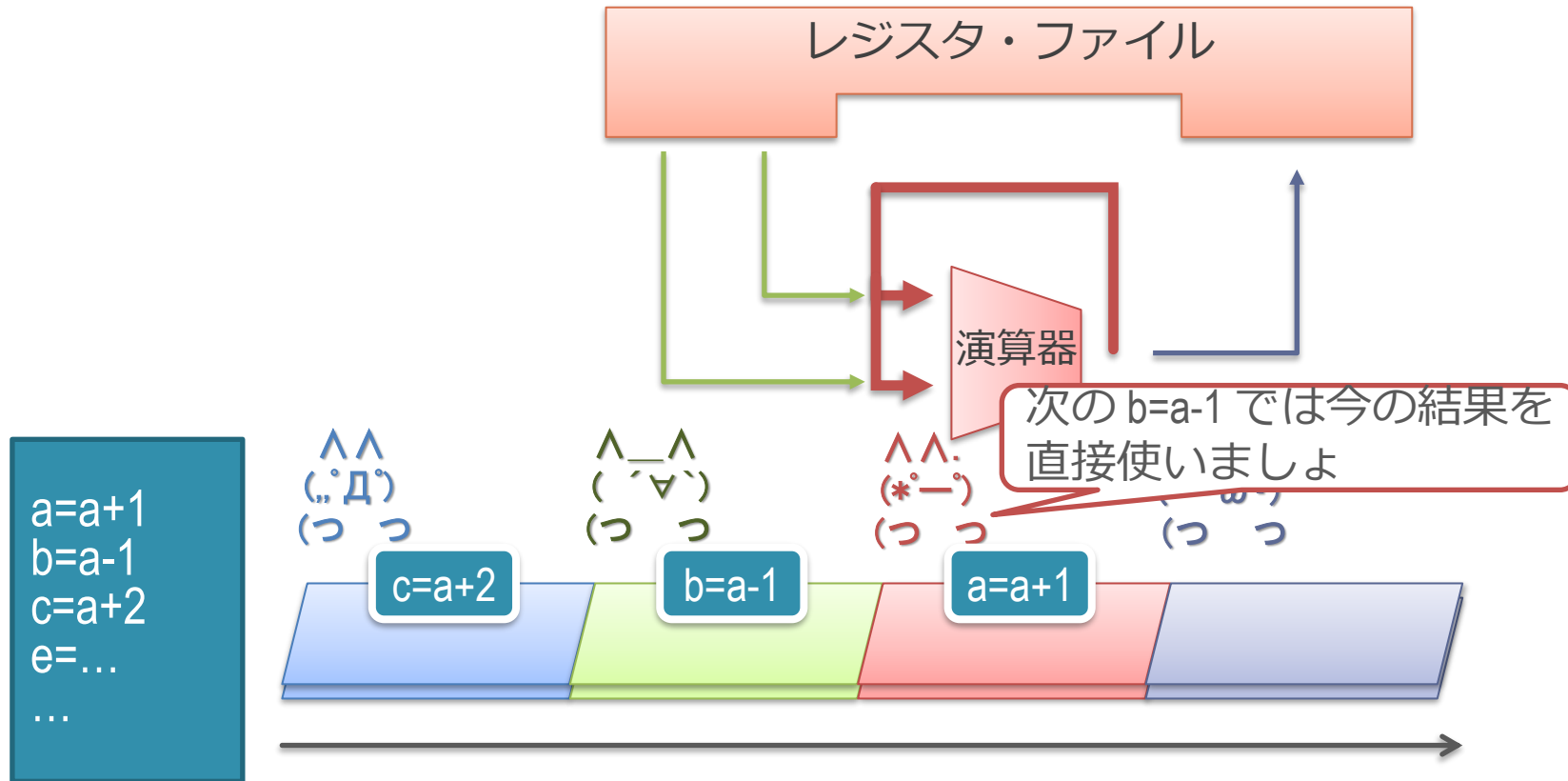
- 欠点2：並列してできる命令が常にあるとは限らない
 - NOPを入れるしかなくなる
 - 実質ストールしてバブルを入れるのと同じになってしまう

データ・ハザードの解消方法

■ 解消方法

1. ストールさせる
2. 遅延スロット（なにもしない）
3. **フォワーディング**

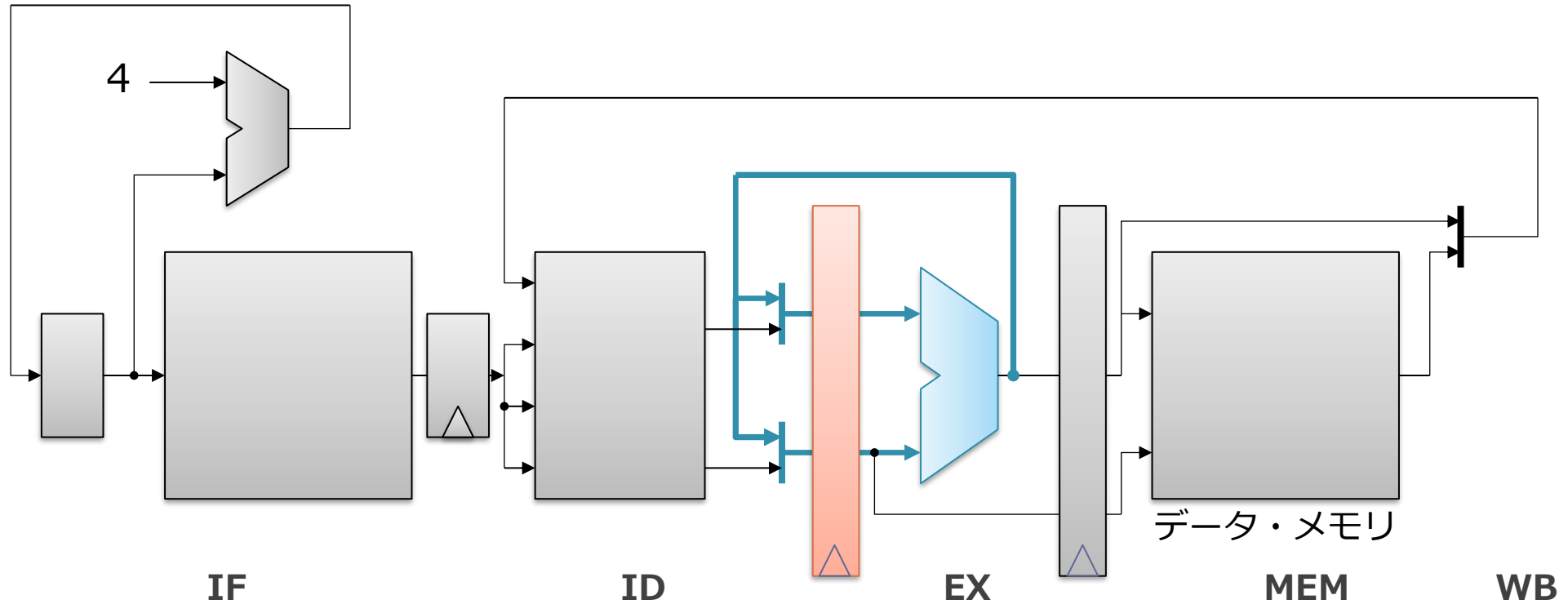
フォワーディング



■ フォワーディング (バイパスとも呼ぶ)

- $(\overset{\wedge}{*}\overset{\circ}{-})$ の人が、次のサイクルに自分の計算結果を即座に使えるよう、手元にも結果を置いて使う
- $(\overset{\wedge}{\prime}\overset{\vee}{})$ がレジスタ・ファイルから読んできた値は必要に応じて捨てる

フォワーディングの回路



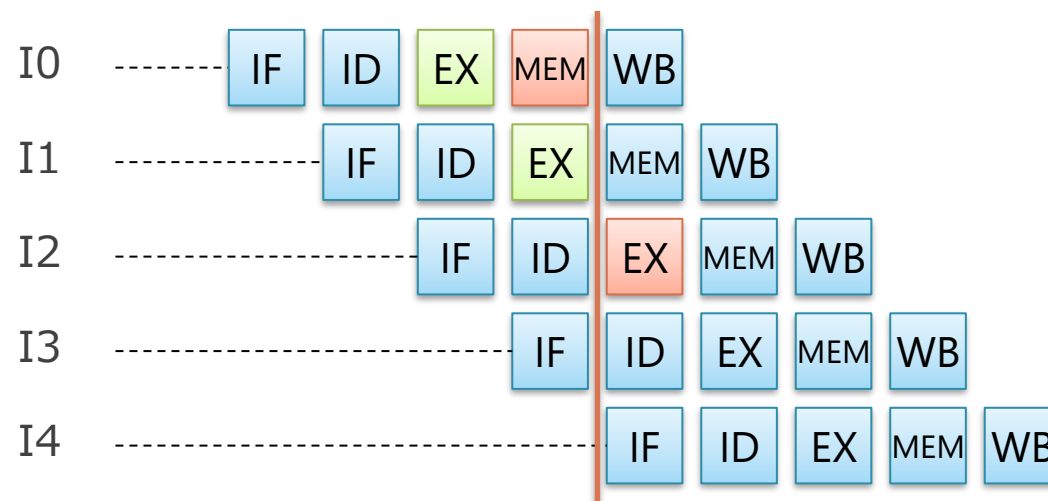
- 演算器の結果を, フィードバック
 - レジスタ・ファイルからの読み出し結果と選択して入力に

フォワーディングの利点

■ 利点：

- 依存関係がある命令が連続できてもパイプラインを動かし続けられる
- バブルを発生させることがない

ロードについては、完全に解決はできない



- ロードではデータ・メモリを読むまでその値は取れない
 - 次の命令は、MEM より後に EX がこないといけない
 - I1 は、I0 のロード結果が見えない
 - この部分はストールや遅延スロットでなんとかすることがおおい

制御ハザード

1. 構造ハザード
2. 非構造ハザード：バックエッジに由来
 - a. データ・ハザード
 - b. 制御ハザード**

分岐命令の処理と制御ハザード



- 「if a > 0」の結果は最終段の(´・ω・)の人まで反映出来ない
 - 先頭は次に a=a+1 と a=a-1 のどちらを取り込めばいいのかわからない

制御ハザードの解消方法

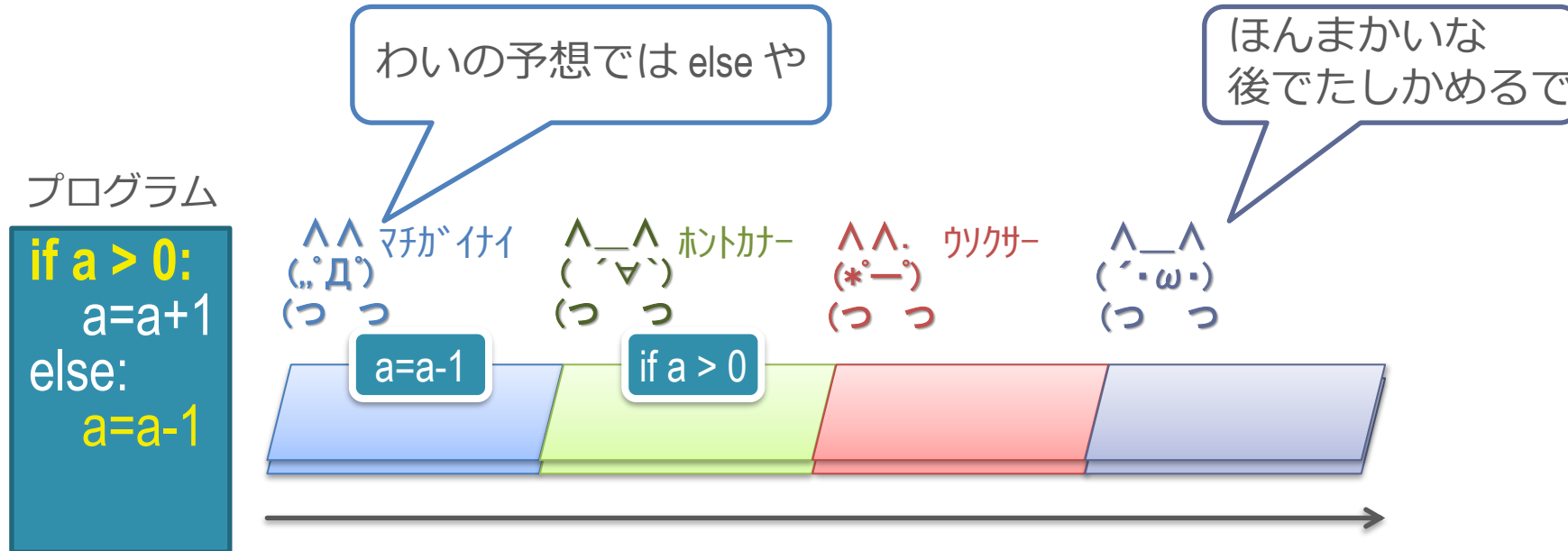
■ 解消方法

1. ストールさせる
2. 遅延スロット（なにもしない）

- 上記は基本的にデータ・ハザードと同様にして適用できる
 - ただしフォワーディングは、制御ハザードでは意味的に無理

3. 分岐予測による投機実行

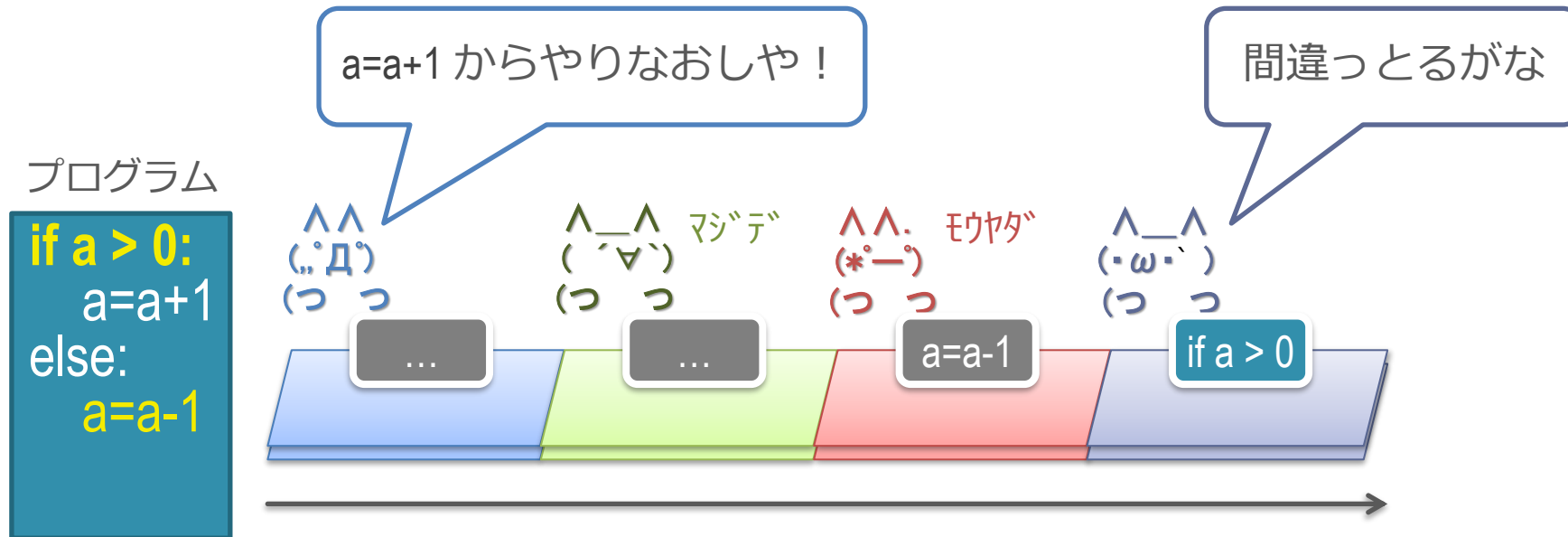
分岐予測



■ 動作

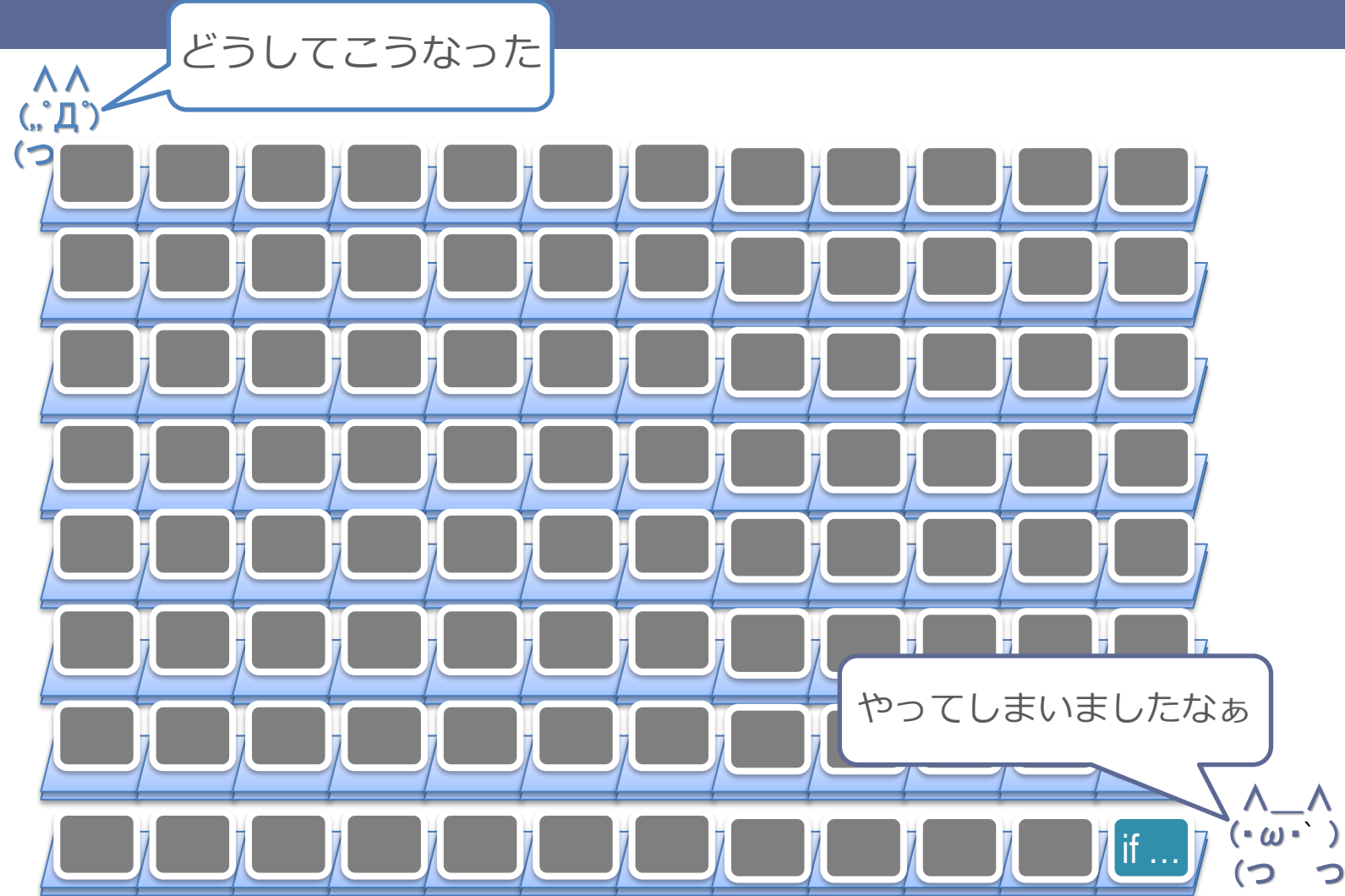
- 「if a > 0」の結果を予測して、命令を取り込む
 - 前はこっちに行ったので、次もこっちに違いないとかで予測
- あとから予測が正しいか確認する

分岐予測ペナルティ



- 予測が間違っていた場合，以降の処理を取り消してやり直す
- この図では，無駄になるのは3命令分

大規模な高性能プロセッサの場合

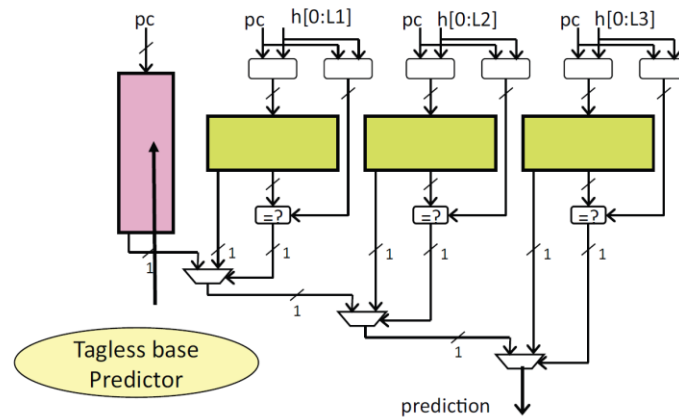


- 取り消しは最悪数十命令以上に
 - Apple の M4P という CPU だと 10命令同時 × 10数段

予測アルゴリズムの基本

- 単純な方法では命令アドレスごとに、過去の分岐方向を憶えておく
 - 基本的には、過去に起きたことが繰り返されると想定
 - if 文は大抵同じ方向に行っている
 - これだけでも8割から9割は当てられる
- より高度な方法では、if 文に到達するまでの経路と if 文の相関を捉える
 - 「あの関数呼んで、あそこの if 文で else だったなら、ここは成立」

分岐予測のアルゴリズムの発展



- André Seznec 先生が作った TAGE という予測器が現在最も予測精度が高いと言われている

- A. Seznec and P. Michaud. A case for (partially)-tagged geometric history length predictors, *Journal of Instruction Level Parallelism* (<http://www.jilp.org/vol8>), 2006
- 写真は <https://team.inria.fr/pacap/members/andre-seznec/> より

- Championship Branch Prediction という大会がある

- 分岐予測は, この大会で出た様々なアイデアを元に発展してきた
- Seznec 先生がもうずっと優勝し続けていた (20年以上)
- 2025 年に久しぶりに開催

CBP2025 Opening Remarks

Sixth Championship Branch Prediction (CBP)

Co-located with ISCA 2025 – Tokyo, Japan

Rami Sheikh
Organizing Committee Chair

June 21, 2025



arm

去年はうちが優勝

CBP2025 Winners ...

Nagoya Institute of Technology
The University of Tokyo

CBP2025 Winner



1st Place \$ prize: \$1,000



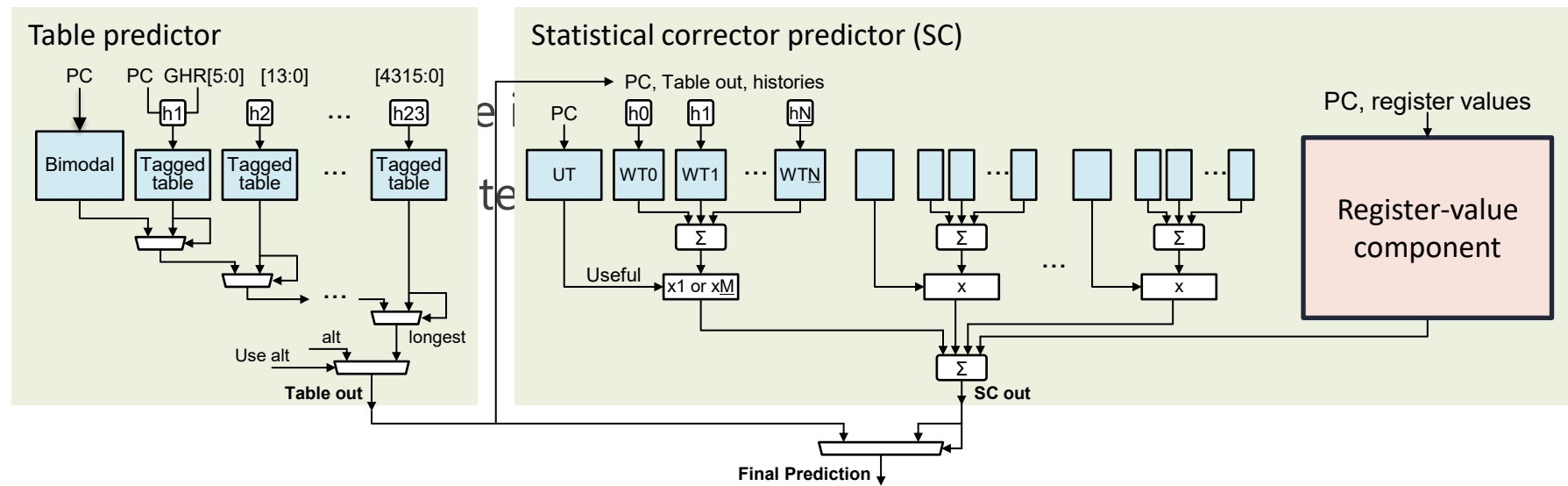
arm



RUNLTS Predictor

我々の世界最強の分岐予測器（2025年6月現在）

- 基本的には TAGE 予測器の構造を踏襲
- レジスタの値と分岐の方向の相関を拾う方式を新しく提案

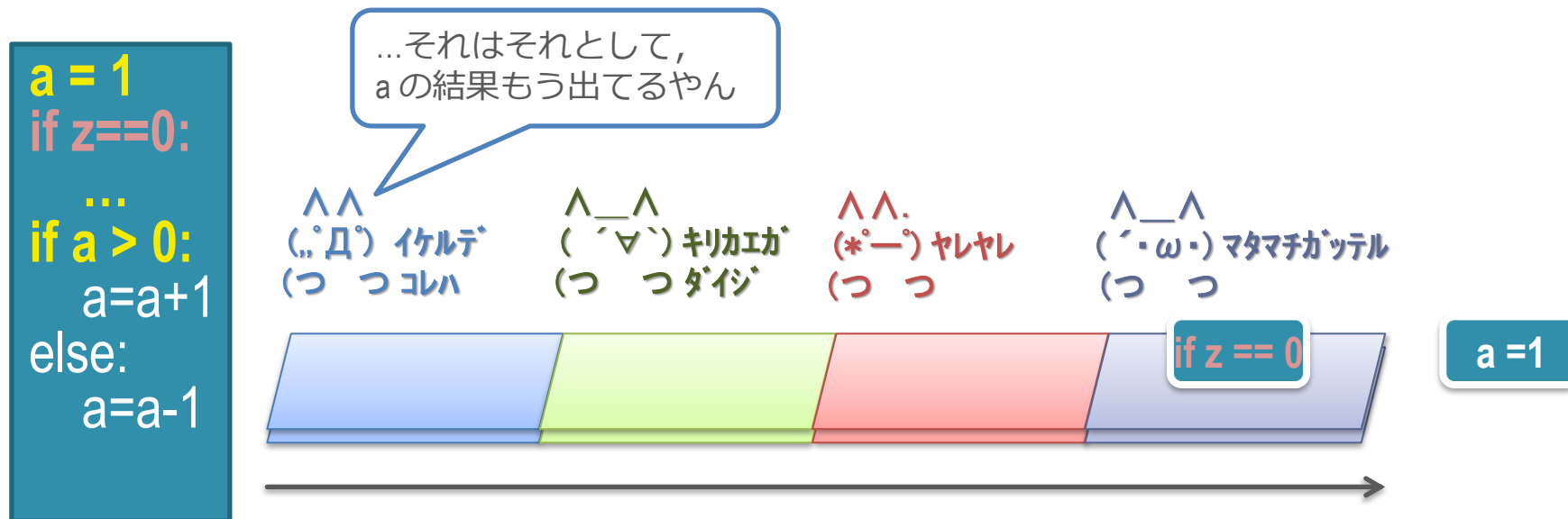


分岐命令の処理と制御ハザード



- 「if a > 0」の結果は最終段の('・ω・)の人まで反映出来ない
 - 先頭は次に a=a+1 と a=a-1 のどちらを取り込めばいいのかわからない

RUNLTS のやりかた



- 連続した分岐の1個目（「if z==0」）でミスがあった場合を考える
 - 「if z==0」の後には一旦取り消される
 - このときしかし、その前に決定された値「a=1」が観測可能
 - これを使う

パイプライン化の限界

- 速度が上がらなくなる理由：
 1. 回路的な理由による周波数向上の限界（前回の講義）
 2. **アーキテクチャ的な理由（ハザード）による実効性能の限界**

アーキテクチャ的な理由による実効性能の限界

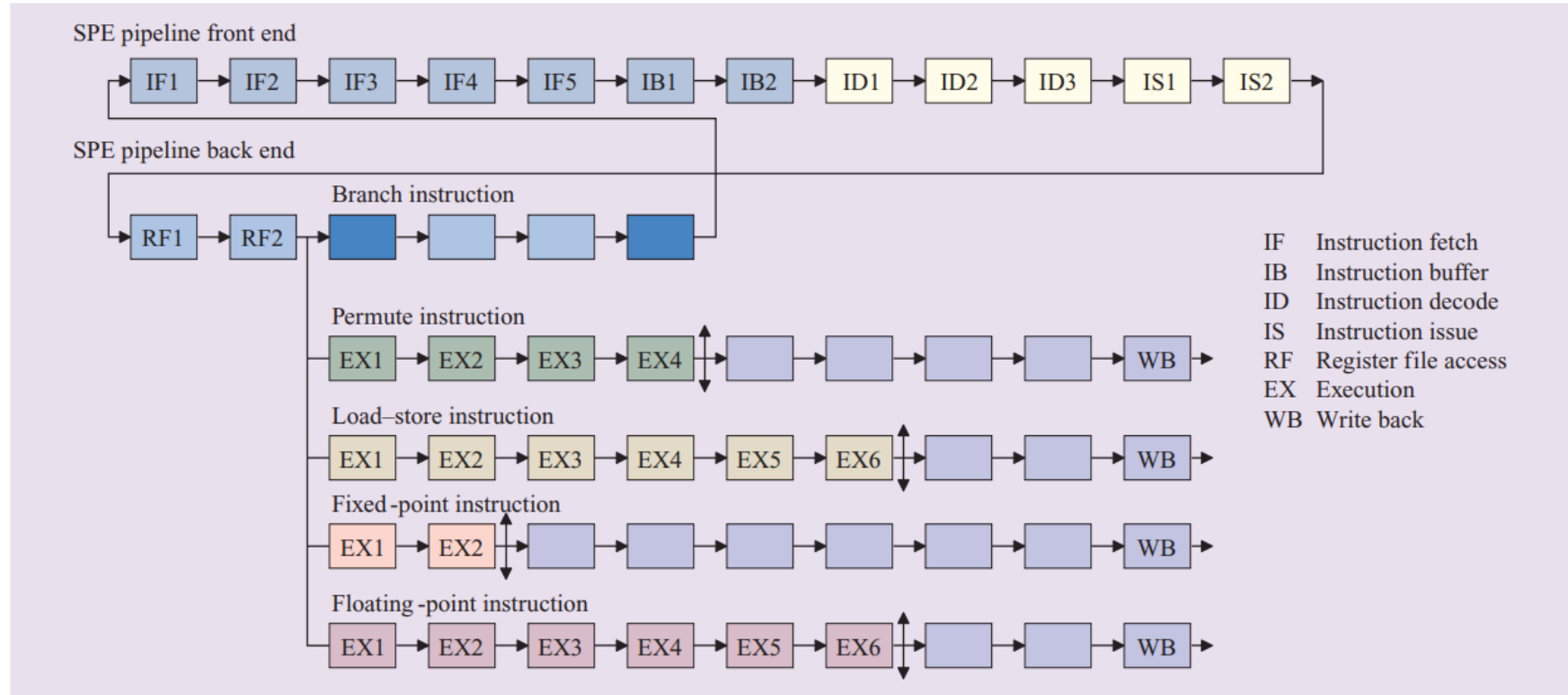
- バックエッジがないパイプライン
 - （回路的な限界にあたるまでは
 - パイプライン段数を増やせば増やすほど性能（周波数）が上がる
- バックエッジがあるパイプライン
 - パイプライン段数を増やすと,
 - 周波数そのものは上がる・・・が,
 - 場合によって、命令を処理できる実効的な速度が落ちる
 - 特に分岐予測ミス・ペナルティによる性能低下が大きい

余談：実際の CPU のパイプライン段数

- 現在は大体 15 ～ 20 段
- Intel Pentium4 (Prescott) 31 段
 - 2004年発売で 3.8 GHz
 - おそらく、歴史上最大の段数
 - 熱くなりすぎ & 性能が出ずで、この後ステージ数は減少
- AMD Zen : 19 段
 - 2017年発売で 4.2GHz

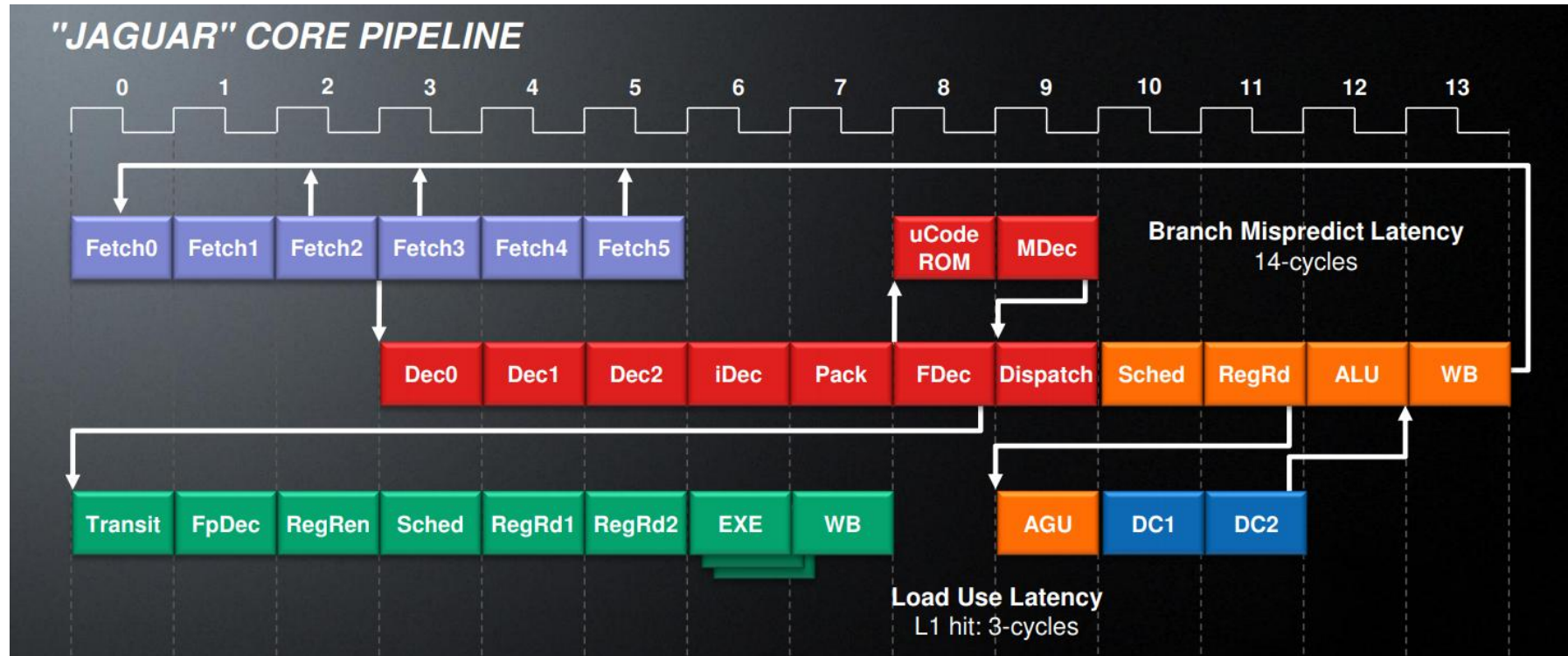
Sony/IBM/東芝 Cell (SPE)

Cell Broadband Engine Architecture and its first implementation—A performance view より



AMD JAGUAR

"JAGUAR" AMD's Next Generation Low Power x86 Core より



ARM Cortex-A15

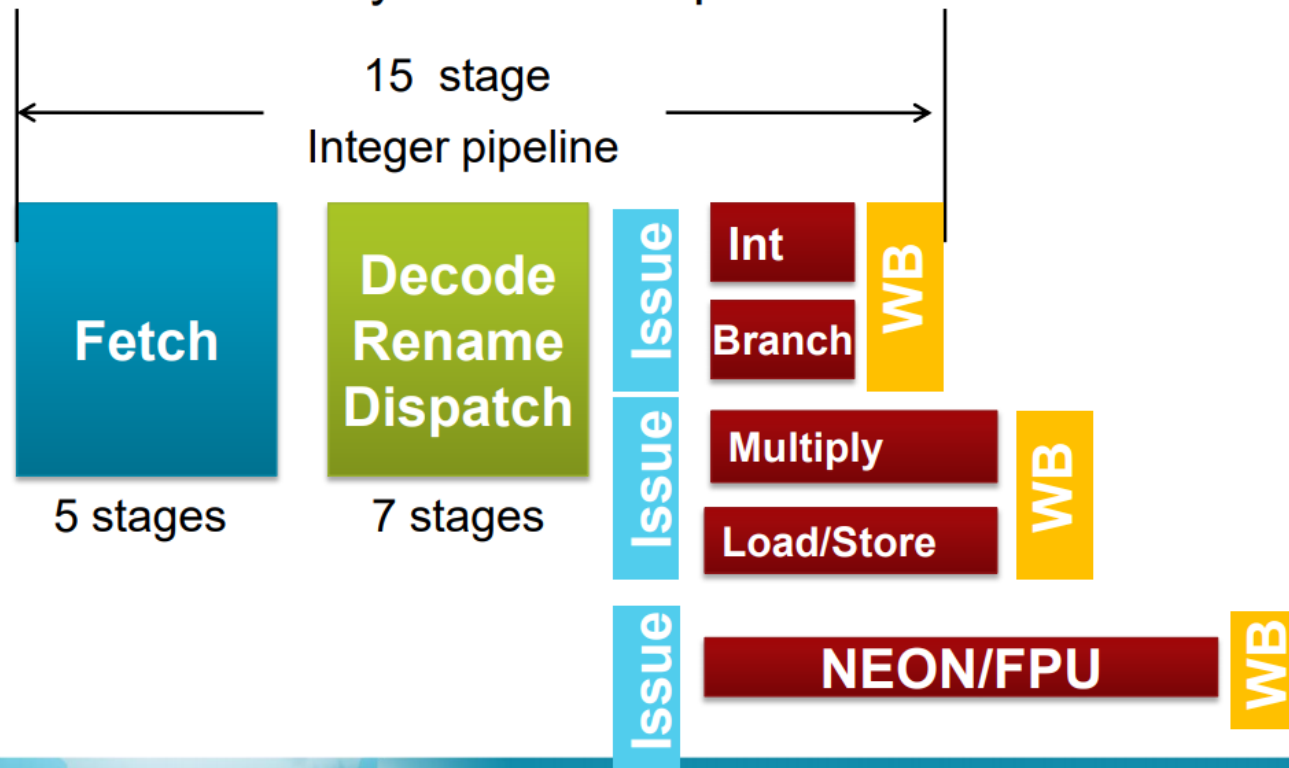
Exploring the Design of the Cortex-A15 Processor

ARM's next generation mobile applications processor より

Cortex-A15 Pipeline Overview

15-Stage Integer Pipeline

- 4 extra cycles for multiply, load/store
- 2-10 extra cycles for complex media instructions



まとめ

1. 構造ハザード：ハード資源の不足に起因
 1. 構造ハザードとはなにか？
 2. その解決方法
2. 非構造ハザード：バックエッジに由来
 - a. データ・ハザード
 - b. 制御ハザード

課題 6

- RISC-V の「add x1←x2,x3」命令を 2 進数で表記すると以下の通りとなる
0000000 00011 00010 000 00001 0110011
(rd←rs1,rs2 として考える)
 - (1) 上記を sub x1←x2,x3 に書き換え, 2 進数と 16 進数の双方で表記せよ
 - (2) 上記を add x2←x3,x4 に書き換え, 2 進数と 16 進数の双方で表記せよ
 - (3) 上記を addi x1←x2,16 に書き換え, 2 進数と 16 進数の双方で表記せよ
- 第 3 回目の講義および次のページの仕様を参考にとすると良い

RISC-V の 基本整数命令

■ 概要

- 加減算，論理演算，ロード・ストア，即値，分岐とジャンプなど
- 各命令は 32bit 幅

RV32I Base Instruction Set							
imm[31:12]			rd		0110111		LUI
imm[31:12]			rd		0010111		AUIPC
imm[20:10:11:19:12]			rd		1101111		JAL
imm[11:0]			rs1	000	rd		JALR
imm[12:10:5]		rs2	rs1	000	imm[4:1:11]	1100011	BEQ
imm[12:10:5]		rs2	rs1	001	imm[4:1:11]	1100011	BNE
imm[12:10:5]		rs2	rs1	100	imm[4:1:11]	1100011	BLT
imm[12:10:5]		rs2	rs1	101	imm[4:1:11]	1100011	BGE
imm[12:10:5]		rs2	rs1	110	imm[4:1:11]	1100011	BLTU
imm[12:10:5]		rs2	rs1	111	imm[4:1:11]	1100011	BGEU
imm[11:0]			rs1	000	rd		LB
imm[11:0]			rs1	001	rd		LH
imm[11:0]			rs1	010	rd		LW
imm[11:0]			rs1	100	rd		LBU
imm[11:0]			rs1	101	rd		LHU
imm[11:5]		rs2	rs1	000	imm[4:0]	0100011	SB
imm[11:5]		rs2	rs1	001	imm[4:0]	0100011	SH
imm[11:5]		rs2	rs1	010	imm[4:0]	0100011	SW
imm[11:0]			rs1	000	rd		ADDI
imm[11:0]			rs1	010	rd		SLTI
imm[11:0]			rs1	011	rd		SLTIU
imm[11:0]			rs1	100	rd		XORI
imm[11:0]			rs1	110	rd		ORI
imm[11:0]			rs1	111	rd		ANDI
0000000		shamt	rs1	001	rd		SLLI
0000000		shamt	rs1	101	rd		SRLI
0100000		shamt	rs1	101	rd		SRAI
0000000		rs2	rs1	000	rd		ADD
0100000		rs2	rs1	000	rd		SUB
0000000		rs2	rs1	001	rd		SLL
0000000		rs2	rs1	010	rd		SLT
0000000		rs2	rs1	011	rd		SLTU
0000000		rs2	rs1	100	rd		XOR
0000000		rs2	rs1	101	rd		SRL
0100000		rs2	rs1	101	rd		SRA
0000000		rs2	rs1	110	rd		OR
0000000		rs2	rs1	111	rd		AND
0000	pred	succ	00000	000	00000	0001111	FENCE
0000	0000	0000	00000	001	00000	0001111	FENCE.I
0000000000000			00000	000	00000	1110011	ECALL
0000000000001			00000	000	00000	1110011	EBREAK
csr			rs1	001	rd		CSRRW
csr			rs1	010	rd		CSRRS
csr			rs1	011	rd		CSRRC
csr			zimm	101	rd		CSRRWI
csr			zimm	110	rd		CSRRSI
csr			zimm	111	rd		CSRRCI

画像は下記より
The RISC-V Instruction Set Manual Volume I: User-Level ISA Document Version 2.2

提出方法

■ 以下を提出：

1. 課題 5

- 提出は Moodleからお願いします
- 画像（紙に書いてスマホで写真でもよし）で提出してください

2. 感想や質問：

- 「感想や質問」のところに投稿してください
- わからない場所がある場合，具体的に書いてもらえると良いです

■ 提出締め切り

- 来週の講義開始まで

■ 注意：

- 課題の出来は，ある程度努力したあとがあれば良しです
- 必ずしも正解していなくても良いです

質問や感想など

質問や感想など

- 課題4でj (jump命令) を使ったのですが、効率悪いですか？
今回の授業に遅刻してしまい、前回課題の解説を聞きそびれてしまったので既におっしゃっていたらすみません。

＜自分の解答＞

- li 10 → A
LABEL1:
b A<=3, LABEL 2
sub A ← A-1
j LABEL1
LABEL2:

- そもそも合ってますか...？

- 課題をやっていて思ったのですが、P型のみ、もしくはN型のみを直列、並列に接続することでNOT、AND、OR回路を作ることば可能なのでしょうか。

- D-FFの回路の話で、Dラッチを2つ繋げていましたが、どうして右側はループの形になっているんですか？左側のループでdから得た値を反転させるだけなら右側はループの他に方法があるんじゃないかと思いました。

質問や感想など

- 命令パイプラインのところに出てくる

^^
(.D)
っ っ

がかわいかったのでいつもより集中して講義資料を見られました！

- 論理回路の図の描き方について教えてください。P型・N型のゲートを含む回路図を描く際、各スイッチを「繋がっている図」と「繋がっていない図」のどちらで描くべきかの区別がつきません。入力によってP型・N型のON/OFFが反転することは理解しているのですが、図として表す際にはどのような基準で繋ぐ・繋がないを判断すればよいのでしょうか？

- 工場の例が分かりやすく、パイプライン化することで、単位時間あたりに処理できる命令の数が増加することがよくわかりました。ただ、はじめ一人目が作業している間、2人目以降はヒマになってしまうため、全員が最初から同時に作業をし始め、その人数を増やすことで全体の作業スピードは上がると思ったのですが、それはできないのでしょうか。

- 今回の課題で、P型とN型を授業資料の通りに書いたのですが、もっと簡略化してすることはできますか？

質問や感想など

- 最近課題が多すぎてキャパシティを超えている気がします。睡眠時間が真っ先に犠牲となった結果授業中眠くて課題の効率が悪くて睡眠時間が。。。という悪循環に陥っています。どうしたらもっと容量良く活動できるでしょうか。アドバイスを頂けたら嬉しいです。

- N型リレーをVDD側に置いたり、P型リレーをGND側に置いたりするとどうなりますか？

- 私は効率厨なので、一つのパイプラインの処理だけで見るとラッチが開くまで待たないといけないとのことでとても苦痛に感じましたが、全体的に見ると処理量が増えると知り嬉しくなりました。ありがとうございます。

- パイプライン化の話を聞いて、合宿で大人数で水をバケツリレーした時のことを思い出しました。2段、4段と段数を増やしても限界があるように、バケツリレーも人数を増やしすぎると、かえって受け渡し为非効率になり、逆に遅くなった経験があります。

- また前回の授業のことになりますが、私はアルバイトで小学生にマイクラを使ったプログラミングを教えているのですが、その経験からも、子どもたちはマイクラの影響によって情報をより身近に感じていると実感しています。

- 普通のCPUだとクロック周波数はどのくらいなのですか？
- またパソコンが熱くなるのは主にこのパイプラインの放熱のせいだったりしますか。

- パイプラインの段数を増やすと速くなり、増やしすぎると限界が来るとありましたが、実際には最適な段数はどのように決めるのでしょうか。

- パイプラインと聞いて一番最初に思い浮かんだのがLinuxコマンドの'|'だったのですが、これは今回の講義と何か関係ありますか？

- ループする論理回路はあまり見たことがなかったため新鮮だった。アスキーアートみたいな猫可愛かったです。

- スマートフォンなどのバッテリー容量や排熱に厳しい制限がある機器のCPUでは、パソコン用のCPUなどと比べて、意図的にパイプラインの段数を減らすような設計がされる傾向にあったりするのでしょうか

- 第4回までのアセンブリ言語の課題について、値をレジスタBに入れてから計算している時（第4回課題）と入れずにそのまま計算している時（第2回課題）では何か違いがありますか？

- 未だにCPUのポインタとかアドレスとかが分かっていないので、大事そうだしもう一回説明してほしい。

- ちょっと今回の授業には関係ないんですけど、アセンブリ言語ってC言語とかとはそもそも何が違うんですか？何に多く使われているんですか？

- 課題についての質問です。2入力ORをNANDの入力にNOTをつける形で作ったのですが、VDDとGNDがいくつも発生し、回路全体でみるとGNDの下にVDDがあるという事例が起きています。VDDとGNDの間に別のVDD、GNDを置いても大丈夫なののでしょうか？もし既に回答済みでしたら申し訳ありません。

- アセンブリ言語のラベルについて、関数のようなものと誤解していました。LABEL: 以降で字下げされている命令文を一括りにし、b LABEL で呼び出されるようなイメージでした。講義内で付箋のようなものという説明があり、認識を改めることができました。

質問や感想など

- まず前回の課題で、出てきている数字が10と1と3だが10と3だけAとBに格納していて、1は数字のまま扱っているのがなぜなのかわからなかった。
- また、今回の課題の2入力ORゲートでは、前回の資料にANDの入力と出力にNOTをつけるとORが作れると書いてあるが、N型2つを並列接続するだけではできないのか気になった。

- 授業の最初に、課題の解説をしていただけてますが、自分が書いたものと微妙に表現が違うなと思うことがたまにあって、それが間違いなのか、気にしなくてもいい程度の違いなのかがわからず不安です。

質問や感想など

- 本日の講義では、命令パイプラインについてと、そのメリット、仕組みについて学習した。その中で、私はカフェでアルバイトをしているのだが、「パイプライン化により実行時間の短縮（スループットの向上）が実現する」ということが、カフェでの頭の中でのレシピの工程の組み立て方にかなり近いものがあると感じた。例えば、複数の注文（①カプチーノ②キャラメルマキアート③ラテの順番）が一度に入ったとき、なるべく早くご提供するために、

①ミルクのスチーム→エスプレッソショットを落とす

②ミルクのスチーム→シロップを入れる

①を仕上げる

②→エスプレッソショットを落とす

③ミルクのスチーム→.....

というように、少しずつ工程をずらしつつ、少ない機具（ショットを落とすものやミルクをスチームする設備）を常に止めることなく無駄なく稼働させるような流れにするために、時折いくつかの作成を止めておくなどして全体の歩調を合わせながら、効率的に作るようにしている。今回のパイプラインの流れは、この経験と似ており、自分が普段やっている思考の仕方が、コンピュータの基盤の世界でも行われているというのが興味深く、親近感さえ湧いた。

加えて、仕事がすぐに終わったとしてもパプライン・ラッチで時間をかけさせるような仕組みは、通信の信号を自動車として信号機を設置し、赤信号の間は動きをとめさせるようなイメージもわいた。複数ステージ間で信号が混じるのを防ぐためのこのような工夫は初めて知るものであり、新鮮に感じた。

- 質問です。期末テストは具体的に7月の何日に実施予定でしょうか。昨年度の資料から、全13回の講義で13回目に試験という流れだと解釈しましたが、合っていますでしょうか。ご教示いただけますと幸いです。

- 質問：パイプライン化によってCPUを高速化できることを学びましたが、もし理論上無限に細かくステージに分けられた場合、最終的にはどのような問題が最も大きなボトルネックになるのでしょうか？

- 講義でパイプライン化の例として、工場の分業が挙げられていたが、4人が同時に最初から最後まで完全に並列で作業を行うのと、パイプラインとでは、なぜ後者の方が優れているのか、という疑問を持った。各命令の階層（I0,I1など）ごとに得意な作業が異なり、1つの作業に特化することで作業効率が得られるということかなと考えたが、どうなのでしょう？